Memory Hierarchy

DSC 102: Systems for Scalable Analytics

Haojian Jin

UC San Diego

Important Information About Your Spending Ability

American Express <AmericanExpress@welcome.aexp.com>
to me

Thu, Aug 3, 2017, 6:41AM

Important information regarding your recent activity

HAOJIAN JIN

Account Ending: 02005

View Account | **Recent Activity**

Transaction Declined

Dear HAOJIAN JIN,

We were unable to authorize the below listed purchase on your American Express Card referenced above.

Reason: The transaction would have caused your balance to exceed your credit limit.

Merchant	Amazon Web Services
Transaction Amount	**$43,845.21**
Balance* (including transaction above)	$44,140.00
Credit Limit	$15,000.00

To use your Card without interruption please spend within your credit limit. You may make a payment to bring down any existing balance.

Make a Payment

or

Call us at 1-800-472-9297

As a reminder, the Check Spending Power tool is available to verify if a future charge will be approved.

copyij vs. copyji: Copy a 2048×2048 integer array

```
void copyij(long src[2048][2048], long dst[2048][2048]) {
    for (long i = 0; i < 2048; i++)
        for (long j = 0; j < 2048; j++)
            dst[i][j] = src[i][j];
}

void copyji(long src[2048][2048], long dst[2048][2048]) {
    for (long j = 0; j < 2048; j++)
        for (long i = 0; i < 2048; i++)
            dst[i][j] = src[i][j];
}
```

Same number of copies. Same result. Which one is faster?

copyij vs. copyji: Results

`copyij` : **4.3 ms** `copyji` : **81.8 ms**

- **~20×** slower, just by swapping two loops
- Nothing in the code "looks" more expensive

Try it yourself: `copy_demo.c`

```
cc -O0 -o copy_demo copy_demo.c && ./copy_demo    # no optimization
cc -O2 -o copy_demo copy_demo.c && ./copy_demo    # optimized
```

2048×2048 `long`, Apple M4 Pro, clang 17

Build	copyij	copyji	Gap
`-O0` (every variable lives in memory)	4.7 ms	29.5 ms	6×
`-O2` (registers + SIMD vectorization)	**0.6 ms**	38.9 ms	**62×**

- `-O2` copies copyij's row **64 bytes per step**; copyji's column can't be vectorized
- The compiler makes the good order faster, but **cannot rescue the bad order**

Try it yourself: `copy_demo.py`

Python + NumPy, 2048×2048 int64, Apple M4 Pro

Version	copyij	copyji	Gap
Pure Python loops	234 ms	389 ms	1.7×
NumPy, row-major (C order)	0.9 ms	49.3 ms	**55×**
NumPy, column-major (Fortran order)	45.9 ms	0.8 ms	**flipped**

Questions for this lecture

- How to measure the speed of a program?
- Why is C faster than Python? Why is NumPy faster than Pure Python? Why is Numpy slightly faster than unoptimized C?
- Why is copyij faster than copyji?
- How can we make copyij even faster?

By the end of this lecture, you need to be crystal clear on the answer to each of these questions.

Q1: How to measure the speed of a program?

Speed = time to finish a workload

- **Wall-clock time** (`real`): what the user waits for
- **CPU time** (`user` + `sys`): time the CPU actually spent on your program
- **Latency**: time for one task · **Throughput**: tasks per second

```
$ time ./copy_demo
real 0.40    user 0.22    sys 0.00
```

- But the copy itself took **0.7 ms**: the rest is setup, checks, and repeats
- **Measure the part you care about**

Measuring in practice

```
t0 = time.perf_counter()
copyij(src, dst)
elapsed = time.perf_counter() - t0
```

- Run several times; report the **min or median** (the first run is "cold")
- Keep the machine quiet (close the browser, no other jobs)
- Always report the **hardware**: same code, different machine → different number

The software view: count cycles

Time = Instructions × Cycles per instruction × Seconds per cycle

- **Instructions**: the algorithm, the language, the compiler
- **Cycles per instruction**: how well the CPU executes them
- **Seconds per cycle**: the clock rate (M4 Pro ≈ 4.5 GHz → **0.22 ns** per cycle)

copyij and copyji, counted in cycles

2048 × 2048 = **4.2 M copies**, clock ≈ 4.5 GHz

Version	Time	Cycles per copy
copyij `-O0`	4.7 ms	~5
copyij `-O2`	0.6 ms	**~0.6**
copyji `-O2`	38.9 ms	**~42**

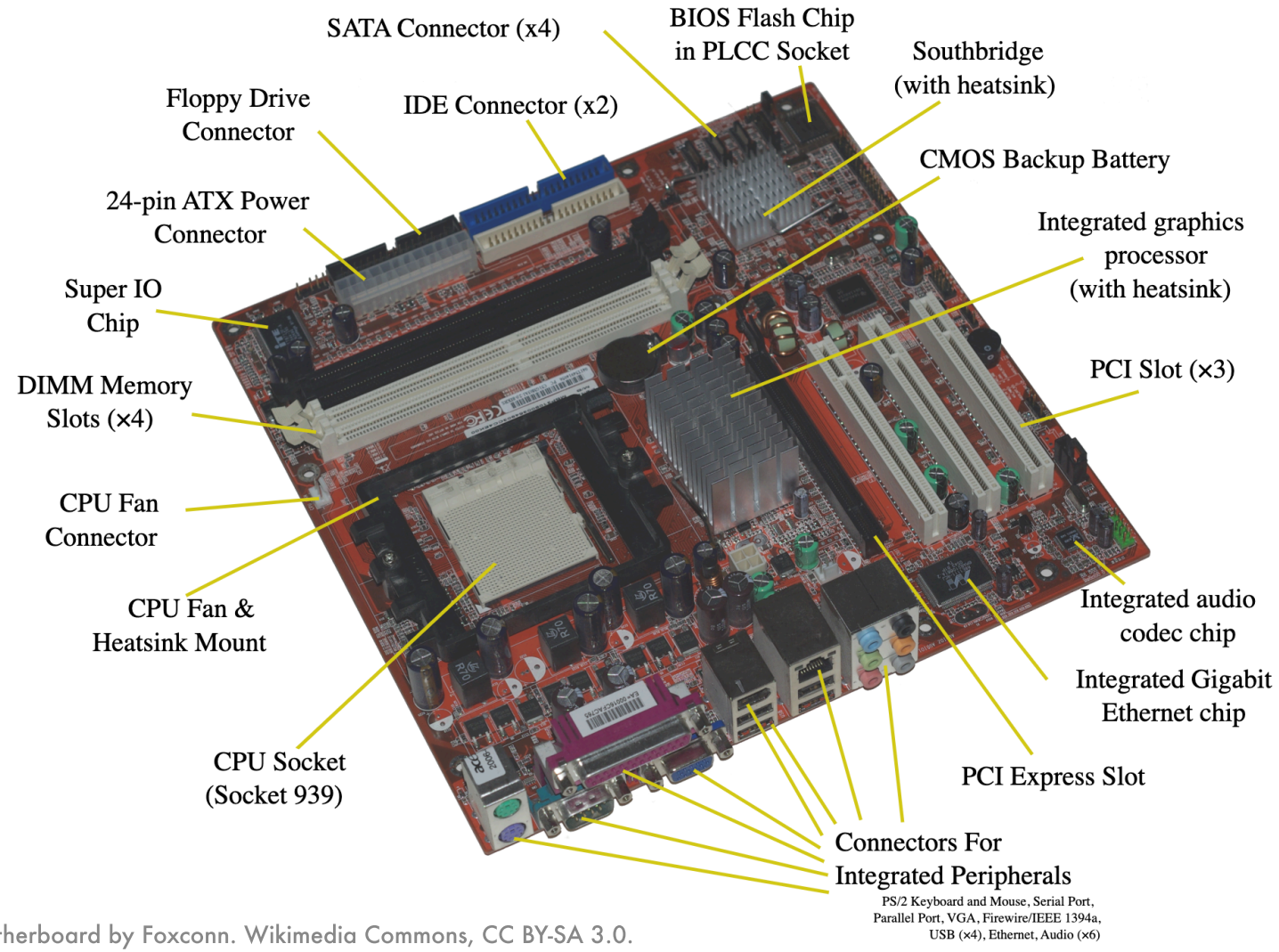
- < 1 cycle per copy: the hardware does **several copies at once**
- 42 cycles per copy: the CPU is mostly **waiting for memory**
- Same instructions, 70× difference → **the hardware matters**

The hardware view: what limits speed?

- **Compute**: operations per second = clock × cores × operations per cycle
- **Parallelism**: can the work be split across cores, SIMD lanes, GPUs, machines?
- **Data movement (I/O)**: how fast data arrives from memory, disk, network
- A program runs at the speed of its **bottleneck**
  - compute-bound vs. memory-bound vs. I/O-bound

To reason about this, we need to know what is inside a computer.

What's inside a computer?

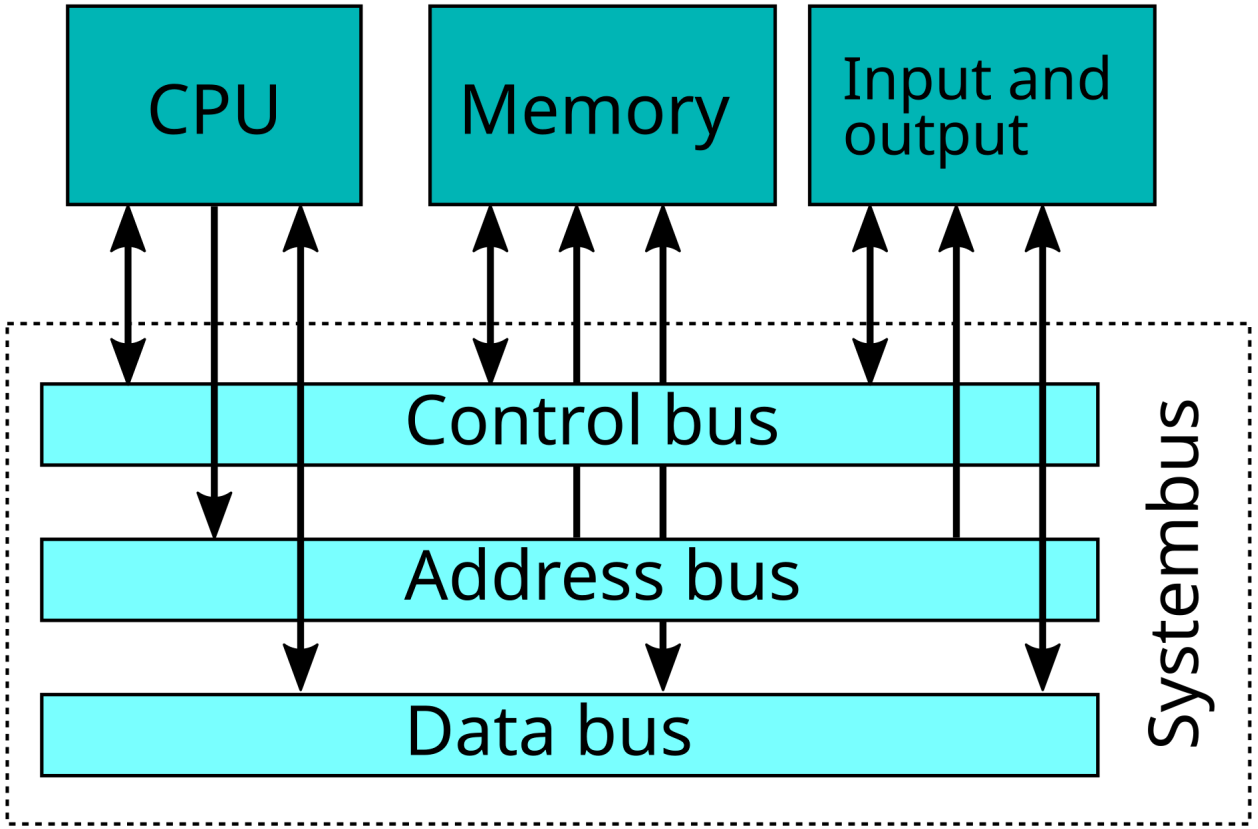


Moxfyre. Acer E360 Socket 939 motherboard by Foxconn. Wikimedia Commons, CC BY-SA 3.0.

Key parts of computer hardware

- **Processor** (CPU, GPU, etc.)
  - Orchestrates and executes *instructions* to manipulate *data*, as specified by a *program*
- **Main memory** (DRAM, Dynamic Random Access Memory)
  - Stores *data* and *programs* for very fast retrieval; byte-level *addressing*
- **Disk** (secondary / persistent storage)
  - Like memory, but *persistent*, *slower*, and higher capacity per dollar
- **Network interface controller** (NIC)
  - Sends and receives data over a network of computers
- **Buses**: the wires that connect them

How the components connect



W Nowicki. Computer system bus. Wikimedia Commons, CC BY-SA 3.0.

Inside the processor

- **ALU**: does the arithmetic; **control unit**: decides what runs next
- **Registers**: a few hundred bytes, the fastest storage
- **Caches** (L1/L2/L3): small, fast copies of recently used memory
- **Cores**: several independent processors on one chip
- **SIMD**: one instruction on several values at once
  - copyij `-O2` moves **64 bytes** per step

My laptop: Apple M4 Pro, 14 cores (10 performance + 4 efficiency), 48 GB memory

Every component has a speed

Component	Latency (one access)	Bandwidth	Typical capacity
Register	~0.3 ns (1 cycle)	—	~KB
L1 / L2 cache	1–5 ns	~1 TB/s	KB – MB
DRAM	~100 ns	100–500 GB/s	16 GB – 2 TB
NVMe SSD	~100 µs	3–14 GB/s	1 – 30 TB
HDD	~10 ms	~0.2 GB/s	10 – 30 TB
Network (same datacenter)	~0.5 ms	1–50 GB/s	—

- One DRAM access ≈ **400 CPU cycles**
- One HDD access ≈ **100,000 DRAM accesses**

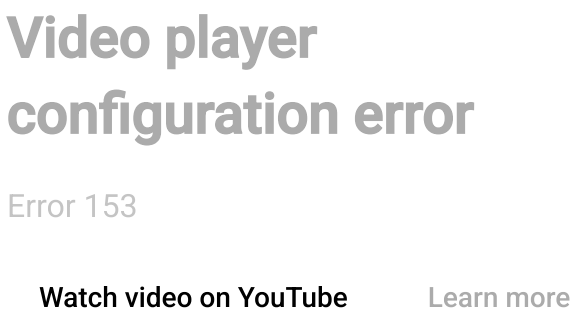
Order-of-magnitude numbers. After J. Dean, "Latency Numbers Every Programmer Should Know."

Keeping the processor busy

- A CPU can do ~**100 G** operations per second
- If each operation reads one 8-byte number → needs **800 GB/s** of data
- What can deliver it?
  - DRAM: ~100–500 GB/s
  - SSD: ~7 GB/s
  - HDD: ~0.2 GB/s
- → Most of the time, the processor is **waiting for data**

CPU vs. GPU

CPU vs. GPU: a demonstration



Error 153

Watch: how does each one paint the picture? · youtube.com/watch?v=gJ84mx3HbY8

Different design goals

CPU

- **Few, powerful cores** (8 – 192)
- Large caches, complex control
- Optimized for **latency**: finish one task fast
- Good at: OS, databases, pandas, branchy code

GPU

- **Thousands of simple cores**
- Small caches, huge memory bandwidth
- Optimized for **throughput**: many identical tasks
- Good at: matrix multiply, deep learning, graphics

In the video: the CPU robot paints one dot at a time, fast; the GPU fires all the dots at once.

CPU vs. GPU in numbers

	Laptop CPU (Apple M4 Pro)	Server CPU (AMD EPYC 9965)	GPU (NVIDIA H100 SXM)
Cores	14	192	16,896 CUDA cores
Memory bandwidth	273 GB/s	~600 GB/s	**3,350 GB/s**
Memory size	up to 64 GB	up to 6 TB	80 GB
Power	tens of W	500 W	700 W
Price	part of a ~$2,000 laptop	~$15,000	~$25,000 – 30,000

- A GPU trades memory **size** for memory **speed**

When does a GPU help?

- **Yes**: the same operation on lots of data (matrix multiply, image filters)
- **No**: branchy, sequential, or small tasks
- **Hidden cost**: data must first move CPU memory → GPU memory over PCIe (~64 GB/s)

Would copyij be faster on a GPU?

- Copy inside GPU memory: 64 MB ÷ 3,350 GB/s ≈ **0.02 ms**
- Moving the 32 MB array to the GPU first: 32 MB ÷ 64 GB/s ≈ **0.5 ms**, about the same as the CPU

Your laptop vs. a server

Laptop vs. server

	MacBook Pro (M4 Pro)	**Cloud GPU server (AWS p5.48xlarge)**
CPU	14 cores	192 vCPUs (AMD EPYC)
Memory	48 GB	2 TB
Storage	1 TB SSD	8 × 3.84 TB NVMe SSD
GPU	integrated	8 × H100 (640 GB GPU memory)
Network	Wi-Fi, ~1 Gb/s	3,200 Gb/s
Power	~100 W	~10 kW

- Same kinds of parts, just **much more of each**

Why are they different?

Laptop

- **One user**, interactive
- Battery life, heat, weight
- Idle most of the day
- ~$2,000

Server

- **Many users and jobs**, 24/7 → throughput
- Reliability: ECC memory, redundant power, hot-swap disks
- Density and cooling: many servers per rack
- Managed remotely; scaled out over a fast network

How much does it cost?

What does it take to train GPT-6?

- **GPT-6 Astra** (2026): trained on **~100,000 NVIDIA Grace Blackwell (GB200)** GPUs at the Stargate data center in Abilene, Texas
- NVIDIA says the **next** run brings **400,000 GPUs** online, drawing **~1.2 GW**, about a large power plant's output

100,000 GPUs	**90 days**	**120 days**
GPU-hours	216 M	288 M
at $2.50 / hr	**$540 M**	$720 M
at $3.50 / hr	$756 M	**$1.0 B**

- **Why so many GPUs?** 216 M GPU-hours on **one** GPU = **~25,000 years**
- The model and its training state do not fit in one GPU's memory, so they are **split across thousands**
- **400,000 GPUs** at the same rates: **~$2 – 4 B** per run

Jensen Huang / Greg Brockman via PC Gamer; cost and power estimates from Wccftech (Sept 2026). OpenAI has not published the cost.

GPU server: 8× NVIDIA H200 NVL

Description	Quantity	Unit Price	Extended Amount
Exxact TensorEX TS4-190785987 - 4U Dual AMD EPYC 9005 8-GPU MGX Server - 8x NVIDIA H200 NVL	1	$298,357.60	$298,357.60
TS4-190785987 - 4U Dual AMD EPYC 9005, 8-GPU MGX Server  - **CPU Support:** Dual Socket AMD EPYC 9005 (Air cool with up 500W TDP) - **Memory:** 24x DIMM Slots (12x Channels Per CPU) Up To 6400MT/s (1DPC) - **Rear PCIe:** 8x PCIe 5.0 x16 FH/FL Double Wide GPU Slots (by PLX Switch), 4x PCIe 5.0 x16 FH/FL Single Width Slots (by PLX Switch), 1x PCIe 5.0 x16 FH/FL Single Wide Slot (from CPU0) - **Storage:** 8x Front Hot-swap U.2 PCIe 5.0 x4 NVMe (by PLX switch), 2x M.2 PCIe 3.0 x2 2280/22110 - **Networking:** 2x 10GBase-T (RJ-45) Intel X710-AT2 + Dedicated GbE Management/IPMI Port w/ mini Display port - **Power Supply:** 4x (3+1) 3200W 80 PLUS Titanium PSU, 100-127Vac @ 1200w, 200-219Vac @ 3000W, 220-240Vac @ 3200w, C20 Inlets - **Server Depth:** 31.4in / 800mm - **Rackmount Rails:** Included - **TPM Module:** On-Board TPM 2.0 Included - **Power Cables:** Available By Request	1		
AMD EPYC Turin 9555 - 64-Core, 128-Threads, 3.2GHz Base, 256MB Cache - SP5 LGA-6096 - 360W	2		
64GB DDR5 6400MT/s ECC REG Memory Module | *1.536TB Total System Memory*	24		
2TB M.2 NVMe PCIe 4.0 SSD	2		
7.68TB U.2/U.3 2.5in NVMe PCIe 5.0 Enterprise SSD	2		
--- *OS MDADM RAID1: 2x 2TB M.2 NVMe PCIe 4.0 SSD* --- *DATA [1-2]: 1x 2TB M.2 NVMe PCIe 5.0 SSD*	1		
NVIDIA H200 Hopper NVL 141GB Graphics Card - PCIe 5.0 - 600W *Includes 5yr NVIDIA AI Enterprise*	8		
NVIDIA NVLink Bridge - 4-way Passive for H200 Hopper NVL PCIe	2		
Ubuntu Linux 24.04 LTS	1		
Broadcom BCM57414 25GbE SFP28 Dual Port Ethernet PCIe Adapter (No Transceiver Modules Included)	1		
ExxactCare 3-Year Parts & Labor	1		
---------- ACCESSORIES ----------			
6-foot C19 to C20 Power Cable	4		
EDU Discount Applied	**1**		

GPU server: 8× NVIDIA RTX PRO 6000

Description	Quantity	Unit Price	Extended Amount
Exxact TensorEX TS4-190785987 - 4U Dual AMD EPYC 9005 8-GPU MGX Server - 8x RTX Pro 6000 Blackwell Server Edition	1	$194,785.61	$194,785.61
TS4-190785987 - 4U Dual AMD EPYC 9005, 8-GPU MGX Server  - **CPU Support:** Dual Socket AMD EPYC 9005 (Air cool with up 500W TDP) - **Memory:** 24x DIMM Slots (12x Channels Per CPU) Up To 6400MT/s (1DPC) - **Rear PCIe:** 8x PCIe 5.0 x16 FH/FL Double Wide GPU Slots (by PLX Switch), 4x PCIe 5.0 x16 FH/FL Single Width Slots (by PLX Switch), 1x PCIe 5.0 x16 FH/FL Single Wide Slot (from CPU0) - **Storage:** 8x Front Hot-swap U.2 PCIe 5.0 x4 NVMe (by PLX switch), 2x M.2 PCIe 3.0 x2 2280/22110 - **Networking:** 2x 10GBase-T (RJ-45) Intel X710-AT2 + Dedicated GbE Management/IPMI Port w/ mini Display port - **Power Supply:** 4x (3+1) 3200W 80 PLUS Titanium PSU, 100-127Vac @ 1200w, 200-219Vac @ 3000W, 220-240Vac @ 3200w, C20 Inlets - **Server Depth:** 31.4in / 800mm - **Rackmount Rails:** Included - **TPM Module:** On-Board TPM 2.0 Included - **Power Cables:** Available By Request	1		
AMD EPYC Turin 9555 - 64-Core, 128-Threads, 3.2GHz Base, 256MB Cache - SP5 LGA-6096 - 360W	2		
64GB DDR5 6400MT/s ECC REG Memory Module | *1.536TB Total System Memory*	24		
2TB M.2 NVMe PCIe 4.0 SSD	2		
7.68TB U.2/U.3 2.5in NVMe PCIe 5.0 Enterprise SSD	2		
--- OS MDADM RAID1: 2x 2TB M.2 NVMe PCIe 4.0 SSD *--- DATA [1-2]: 1x 2TB M.2 NVMe PCIe 5.0 SSD*	1		
NVIDIA RTX PRO 6000 Blackwell Server Edition - 96 GB GDDR7, PCIe 5.0 x16, 16-pin, Passive Design, **600W Max** TDP	8		
Broadcom BCM57414 25GbE SFP28 Dual Port Ethernet PCIe Adapter (No Transceiver Modules Included)	1		
Ubuntu Linux 24.04 LTS	1		
ExxactCare 3-Year Parts & Labor	1		
---------- ACCESSORIES ----------			
6-foot C19 to C20 Power Cable	4		
EDU Discount Applied	**1**		

H200 NVL vs. RTX PRO 6000

Comparing **one GPU**. RTX here means **Blackwell Server Edition**.

Specification	H200 NVL	RTX PRO 6000
Architecture	Hopper	Blackwell
GPU memory	**141 GB HBM3e**	96 GB GDDR7
Memory bandwidth	**4.8 TB/s**	1.597 TB/s
Peak FP32 compute	60 TFLOPS	**120 TFLOPS**
Maximum GPU power	600 W	600 W

- H200: **1.47× the memory capacity**, **3× the memory bandwidth**
- RTX: **2× the peak FP32 compute** (32-bit floating point)

Advertised peak specifications: NVIDIA H200 NVL; NVIDIA RTX PRO 6000 Blackwell Server Edition.

Performance depends on the workload

Scan **48 GB already in GPU memory**, reading every byte once:

	H200 NVL	**RTX PRO 6000**
Peak memory bandwidth	4,800 GB/s	1,597 GB/s
Ideal time = 48 GB ÷ bandwidth	**10 ms**	**≈30 ms**

- These are **theoretical lower bounds**. Real execution adds overhead.
- A compute-bound FP32 kernel may favor RTX's higher compute rate
- RTX also has **ray-tracing cores** and **native FP4 Tensor Cores**

The bottleneck decides which specification matters.

Calculated from NVIDIA H200 NVL and RTX PRO 6000 specifications. Decimal GB/TB; excludes CPU–GPU transfers.

One job across several GPUs

H200 NVL

- **NVLink** connects groups of 2 or 4 GPUs
- The quote includes **two 4-GPU bridges**
- Each 4-GPU group has **564 GB** of total GPU memory

RTX PRO 6000

- GPU communication uses **PCIe Gen 5**, without NVLink
- Eight GPUs have **768 GB** of total GPU memory
- Independent jobs can use separate GPUs with little communication

Splitting one job across GPUs adds **communication cost**. Each GPU still has its own memory, so the program must distribute the data.

NVIDIA H200 NVL; Lenovo RTX PRO 6000 product guide; H200 server quote earlier in this section.

What does the extra $104,000 buy?

Full server quote	8× H200 NVL	8× RTX PRO 6000
Quoted price	**$298,357.60**	**$194,785.61**
Total GPU memory	1,128 GB	768 GB

- H200 costs **$103,571.99 more**, a **53% premium**
- It provides more GPU memory, higher memory bandwidth, and NVLink
- Better throughput per purchase dollar requires **over 1.53× the measured throughput** on your workload

Which resource limits the job you actually want to run?

Prices: the two Exxact server quotes shown above, including CPUs, RAM, storage, and support. H200 also includes five years of NVIDIA AI Enterprise. Purchase-price comparison excludes operating costs.

GB200 vs. H200

- **H200** (Hopper, 2024): one GPU; 8 per server, air-cooled
- **GB200** (Grace Blackwell, 2025): a "superchip" = **1 Grace CPU** (72 ARM cores) + **2 Blackwell GPUs**; 36 of them form one **NVL72** rack of **72 GPUs**

Per GPU	H200	Blackwell (GB200)	
GPU memory	141 GB	186 GB	1.3×
Memory bandwidth	4.8 TB/s	8 TB/s	1.7×
BF16 (sparse)	1,979 TFLOPS	5,000 TFLOPS	**2.5×**
FP8 (sparse)	3,958 TFLOPS	10,000 TFLOPS	**2.5×**
FP32	67 TFLOPS	80 TFLOPS	1.2×
NVLink	0.9 TB/s	1.8 TB/s	2×
GPUs per NVLink domain	8	**72**	9×

- Big gains in **low-precision** math, bandwidth, and **GPU-to-GPU links**
- Little gain in **FP32**
- Better for huge training runs, but needs **liquid cooling** and a whole rack (~140 kW)

NVIDIA H200 and GB200 NVL72 product pages (per-GPU values = superchip ÷ 2). Rack power: Wccftech, Sept 2026.

Training vs. serving: which GPU fits?

Workload	What does the GPU need to do?	Starting choice
Large training jobs	Store activations, gradients, and optimizer state. Synchronize gradients across GPUs.	**H200 NVL**: more memory and NVLink
Cost-conscious serving	Run forward passes with fixed weights. Model + KV cache + runtime fit in 96 GB.	**RTX PRO 6000**: lower-cost replicas and FP4 support
Low-latency LLM serving	Generate tokens quickly, often limited by memory bandwidth at small batches.	**H200 NVL**: faster memory

Training updates the model. Serving runs an already trained model for users.

Both can train and serve. RTX also suits smaller training and fine-tuning jobs.

Selection guidance inferred from H200 NVL, RTX PRO 6000 Blackwell Server Edition, and the quotes above. See NVIDIA's serving comparison and training memory guide.

Hardware prices (approx., Sept 2026)

Component	Example price	Per GB
Desktop CPU	$300 – 600	
Server CPU (192-core EPYC)	~$15,000	
DRAM (DDR5)	32 GB ≈ $450	**~$15**
NVMe SSD	1 TB ≈ $100	~$0.10
HDD	20 TB ≈ $400	~$0.02
Consumer GPU (RTX 5090)	$2,000+	
Datacenter GPU (H100)	$25,000 – 30,000	

- Faster storage costs more per byte: DRAM ≈ 150× SSD ≈ 750× HDD
- DRAM prices rose several-fold since 2025: AI datacenters buy most of it

DRAM: capitalandcompute.net/memory-prices (Sept 2026). Other prices are typical US retail / list.

Cloud: rent instead of buy

Resource (AWS, us-east-1, on-demand)	Price
General VM: 2 vCPUs, 8 GB (m7i.large)	~$0.10 / hour
8 × H100 server (p5.48xlarge)	~$55 / hour (≈ $6.88 per GPU-hour)
Object storage (S3)	~$0.023 / GB-month
SSD block storage (EBS gp3)	~$0.08 / GB-month
Data out to the internet	~$0.09 / GB

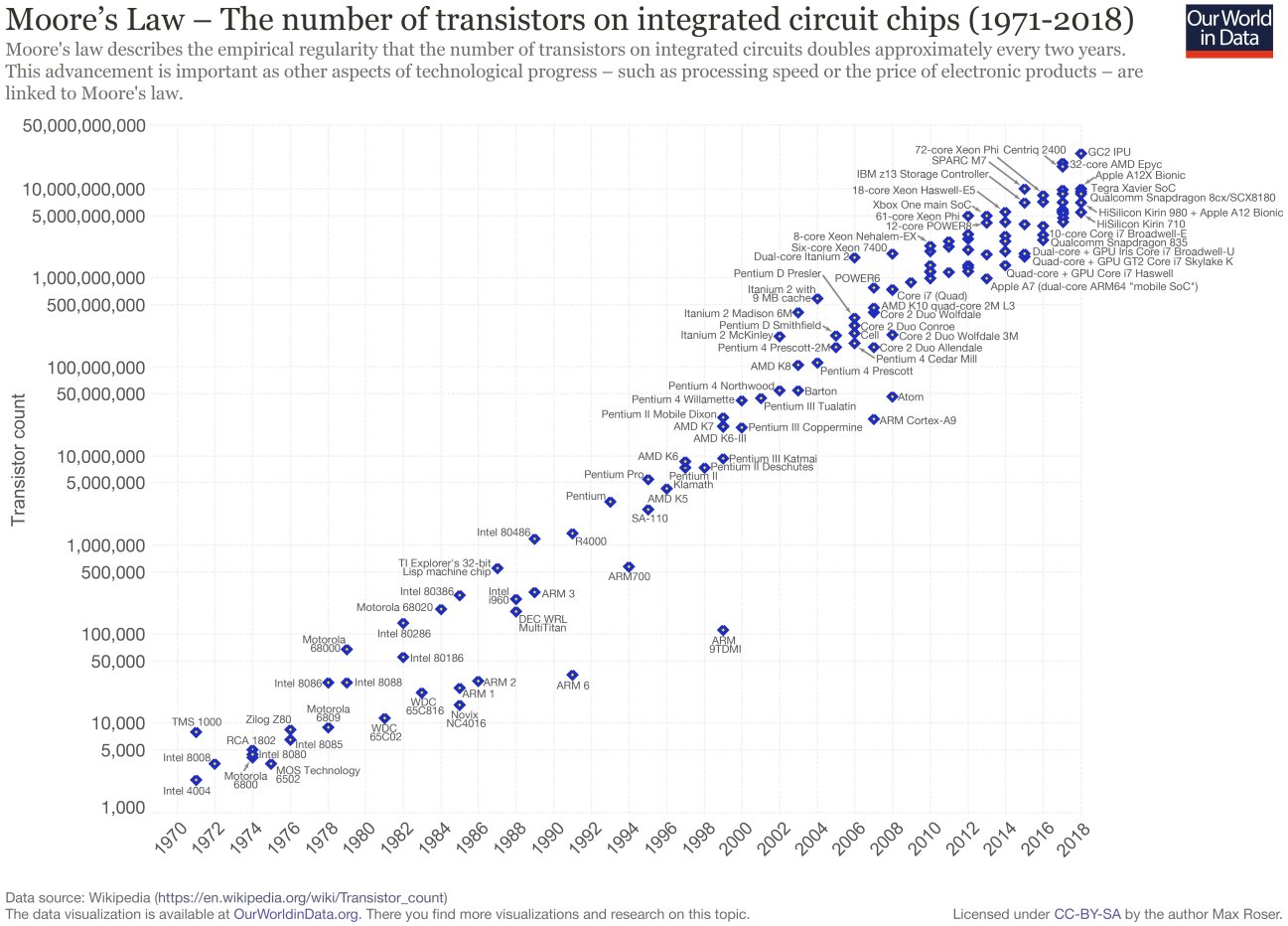
- Smaller GPU clouds rent H100s for ~$2 – 3 / GPU-hour

aws.amazon.com/ec2/pricing; aws.amazon.com/s3/pricing. Prices change often.

Buy or rent? Quick math

- One H100: ~**$27,000** to buy vs. **$6.88 / hour** on AWS
  - Break-even ≈ 3,900 hours ≈ **5 months of 24/7 use**
  - Buying also means a server, power, cooling, networking, and people
- 1 TB for a year: S3 ≈ **$276**; a 1 TB SSD ≈ **$100**
- Downloading 1 TB out of AWS ≈ **$90**
- Rule of thumb: cloud wins for **bursty** work; owning wins for **steady 24/7** load

Moore's Law

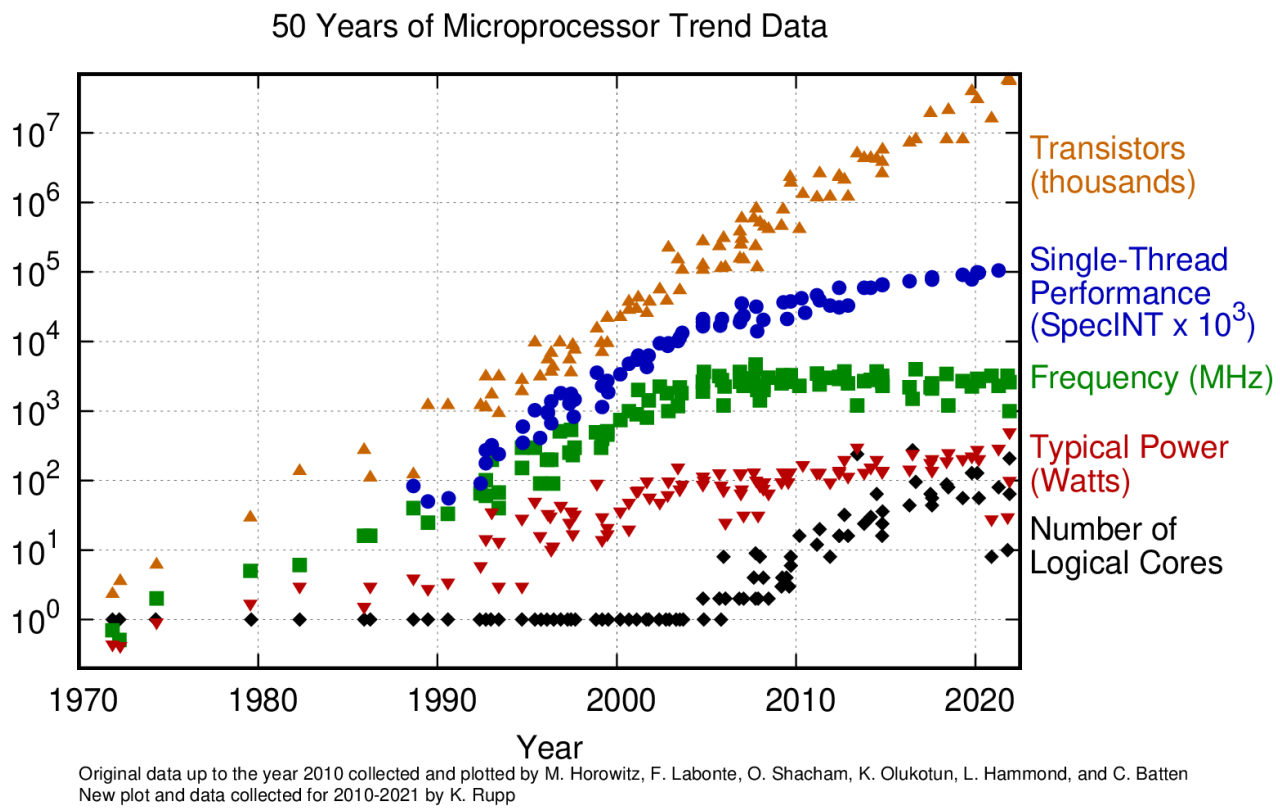


- Roughly **2× as many transistors per chip every two years**
- **Logarithmic y-axis**: equal steps represent equal ratios
- At that rate, ten years gives $\mathbf{2^5 = 32\times}$ the transistors

Program speed still depends on **architecture and software**.

Image: Max Roser / Our World in Data, via Wikimedia Commons (CC BY 4.0). History: Intel.

Transistors, clock speed, and parallelism

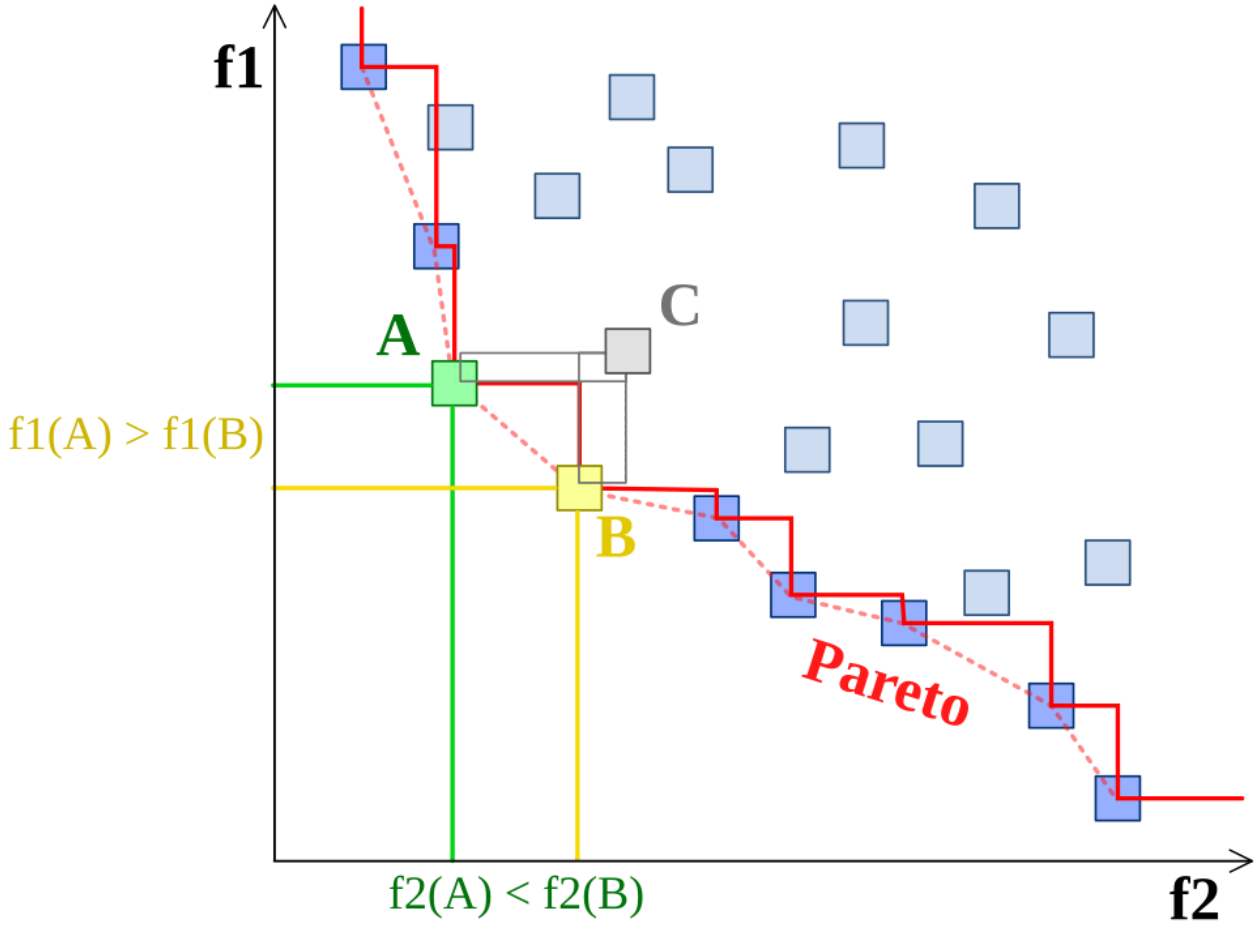


- Transistor counts kept rising as **clock growth slowed** in the mid-2000s
- More transistors can fund **more cores and larger caches**
- Software gains speed through **parallel work** and **data locality**

Why can copyji still spend most of its time waiting for memory?

Image: Karl Rupp, 50 Years of Microprocessor Trend Data, through 2021 (CC BY 4.0). Original data through 2010: M. Horowitz et al.

Pareto dominance and the frontier



Lower f_1 and f_2 are better. Think **latency** and **cost**.

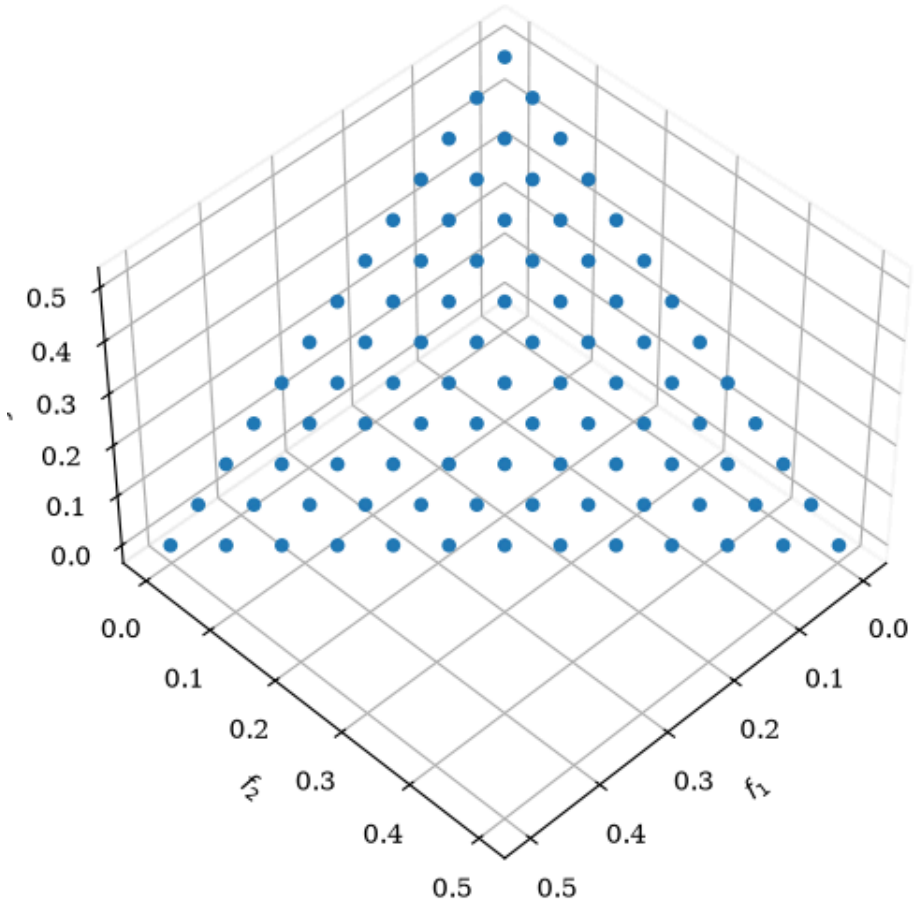
- **A and B both dominate C**
- **A vs. B** trades off the two objectives

Dominates: no worse on any objective, better on at least one.

The **Pareto frontier** contains all non-dominated points.

Image: Johann Dréo, Front pareto (CC BY-SA 3.0). Concept: pymoo, multi-objective optimization.

Three objectives: a Pareto surface



For hardware, consider **latency, cost, and power**.

- **f_3 is the vertical axis**. Minimize all three objectives.
- Non-dominated points can form a **surface**
- Improving one objective requires giving up something on another

Real hardware offers a **discrete set of choices**.

Image: Julian Blank / pymoo, Scatter 3D (Apache 2.0). Synthetic DTLZ1 example, not hardware measurements.

A hardware choice on the Pareto surface

Illustrative numbers, same workload and output quality. Lower is better.

System	Purchase cost	Latency	Power
A	$10,000	12 ms	200 W
B	$15,000	8 ms	300 W
C	$20,000	6 ms	450 W
D	$18,000	10 ms	350 W

- **B dominates D**: cheaper, faster, and lower power
- **A, B, and C** are non-dominated within this set
- With **budget ≤ $15,000** and **latency ≤ 10 ms**, choose **B**

New technology can move the frontier. Our requirements choose a point on it.

Hypothetical classroom example. Compare real systems using measured results for the same workload, software, precision, and quality target.

Pareto surfaces for language models

Optimize **task quality** ↑, **response latency** ↓, and **cost per task** ↓.

Models	Trade-off they target
Claude / GPT-5.x	Strong reasoning and coding; trade thinking time and cost for quality
Gemini Flash-Lite	Low cost and latency for high-volume, simpler tasks
Jev (TypeSafe AI)	Fast decisions; restrict outputs to choices, scores, and probabilities
Thinking Machines Lab interaction models	Real-time audio/video/text collaboration; **responsiveness** matters too

All aim to earn a position on the frontier.

Which are non-dominated depends on the **workload and objectives**.

Conceptual comparison of design goals, not a measured ranking. Sources: Anthropic; OpenAI; Google; TypeSafe AI; Thinking Machines Lab. Checked September 2026.

Q1: How to measure the speed of a program?

- **Measure**: wall-clock time, repeated, on stated hardware
- **Software**: instructions × cycles per instruction × seconds per cycle
- **Hardware**: parallelism (cores, SIMD, GPUs) and data movement (memory, disk, network)
- Speed is set by the **bottleneck**: copyji is memory-bound
- **Cost** is part of speed: $ per hour, $ per GB

Q2: Why is C faster than Python? Why is NumPy faster?

One program, three levels

C source (what we write)

```c
for (long i = 0; i < N; i++)
  for (long j = 0; j < N; j++)
    dst[i][j] = src[i][j];
```

Assembly (ARM, `clang -O2 -S`)

```
ldp  q0, q1, [x12, #-32]
ldp  q2, q3, [x12], #64
stp  q0, q1, [x13, #-32]
stp  q2, q3, [x13], #64
subs x14, x14, #8
b.ne LBB0_6
```

Machine code (what the CPU runs)

```
ad7f0580   ldp  q0, q1, ...
acc20d82   ldp  q2, q3, ...
ad3f05a0   stp  q0, q1, ...
ac820da2   stp  q2, q3, ...
f10021ce   subs x14, ...
54ffff61   b.ne ...
```

- Each instruction is **4 bytes** on ARM
- The CPU only understands the bytes on the left
- A **compiler** translates the top into the bottom

What is ARM? (and x86, RISC-V)

- An **instruction set architecture (ISA)**: the fixed vocabulary of instructions a CPU understands, and how each one is encoded in bits
- Machine code for one ISA **does not run** on another; the compiler's back end targets one ISA at a time

	ARM (AArch64)	x86-64	RISC-V
Designed by	Arm Ltd., licensed to Apple, Qualcomm, AWS, NVIDIA, ...	Intel, AMD	UC Berkeley (2010); now RISC-V International
License	pay Arm to use it	only Intel and AMD build it	**open and royalty-free**: anyone can build one
Where you find it	phones, Apple M-series Macs, AWS Graviton servers	most Windows PCs, most cloud servers	microcontrollers, embedded chips, custom AI accelerators
Style	**RISC**: few, simple instructions	**CISC**: many, complex instructions	**RISC**
Instruction length	**always 4 bytes**	**1 to 15 bytes**	4 bytes (2 with the compressed extension)

Why is every ARM instruction 4 bytes?

ARM `f10021ce` = `subs x14, x14, #8`: every field sits at a **fixed bit position**

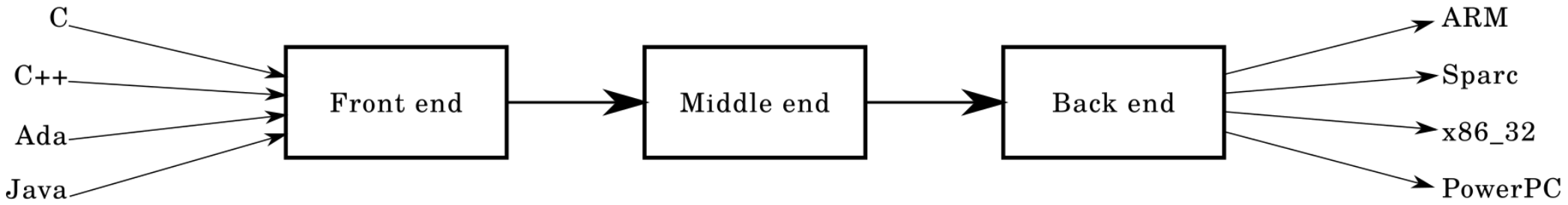
111100010	0	000000001000	01110	01110
opcode: SUBS, 64-bit	shift	constant = 8	source = x14	dest = x14

x86-64, same copyij (`clang -target x86_64`): lengths vary

```
55                              pushq  %rbp                    1 byte
eb 2d                           jmp    0x4c                    2 bytes
48 89 e5                        movq   %rsp, %rbp              3 bytes
48 81 c1 00 40 00 00            addq   $0x4000, %rcx           7 bytes
66 66 2e 0f 1f 84 00 00 00 00 00  nopw   %cs:(%rax,%rax)      11 bytes
```

- **Fixed length**: the CPU knows where every instruction starts, so it can decode many **in parallel**
- **Variable length**: denser code, but the CPU must decode one to find the next
- Cost of fixed length: only 12 bits for a constant; big constants need several instructions

How a compiler is organized



- **Front end**: one per language (C, C++, Rust, Swift, …)
- **Middle end**: one shared optimizer working on an **intermediate representation (IR)**
- **Back end**: one per CPU (x86, ARM, RISC-V, GPU, …)
- N languages × M CPUs needs N + M parts, not N × M compilers

Pronesto. Compiler design. Wikimedia Commons, CC BY-SA 3.0.

Why add these compiler layers?

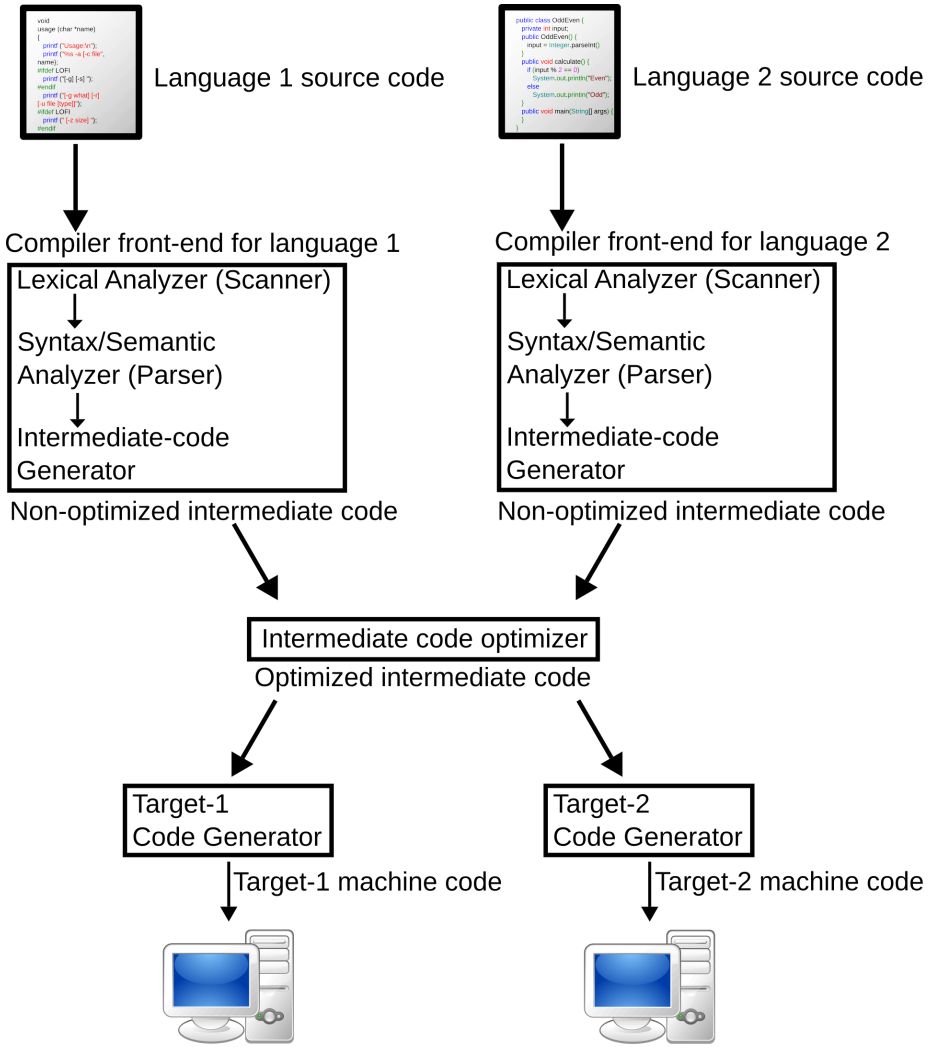
Example: C and Rust, targeting ARM and x86 — both use a common LLVM IR.

Change	What we add or improve	What we reuse
Add a language	A front end that produces the common IR	Optimizer + existing CPU back ends
Add a CPU	A back end that translates IR to its instructions	Existing language front ends + optimizer
Improve optimization	An IR pass, e.g., remove unused calculations	The improvement can benefit multiple languages and CPUs

- **Less duplicated work**: 4 languages × 3 CPUs → 4 front ends + 3 back ends, plus a shared optimizer
- **Easier maintenance**: language syntax and CPU instructions live in separate components
- **Portable source**: recompile for another CPU without rewriting the algorithm

The layers do work during compilation; they do not require an interpreter at

Inside the three phases



Surachit. Compiler. Wikimedia Commons, CC BY-SA 3.0.

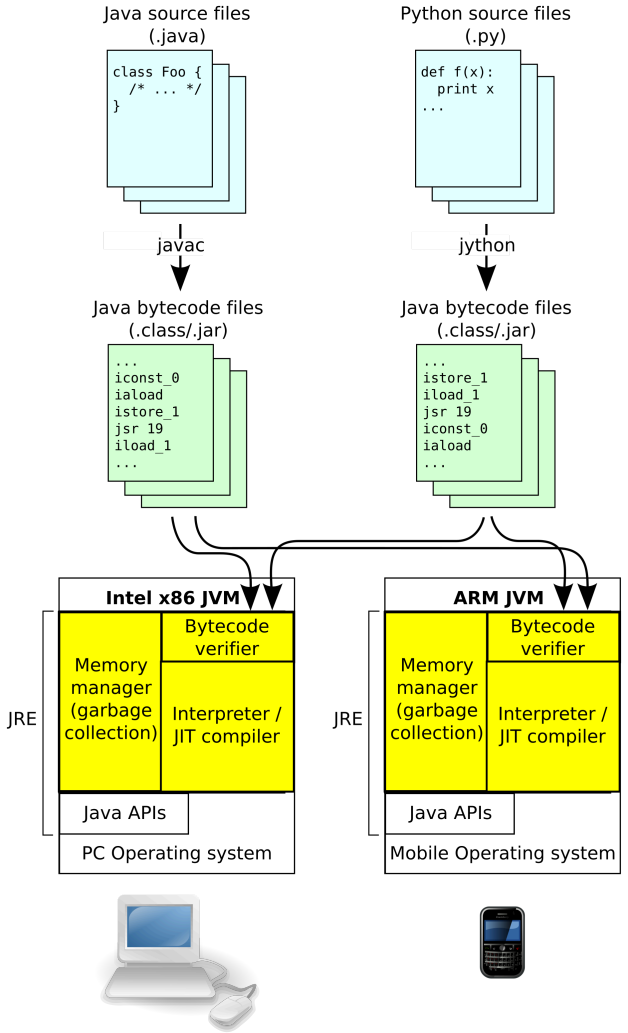
The intermediate representation, for real

LLVM IR for the body of copyij (`clang -S -emit-llvm`), between C and machine code:

```
%17 = getelementptr inbounds [2048 x i64], ptr %15, i64 %16   ; &src[i]
%19 = getelementptr inbounds [2048 x i64], ptr %17, i64 0, i64 %18  ; &src[i][j]
%20 = load i64, ptr %19, align 8                               ; t = src[i][j]
...
%25 = getelementptr inbounds [2048 x i64], ptr %23, i64 0, i64 %24  ; &dst[i][j]
store i64 %20, ptr %25, align 8                                ; dst[i][j] = t
```

- Explicit types (`i64`), loads, and stores, but **no CPU registers yet**
- The optimizer rewrites IR; each back end turns the same IR into x86, ARM, …
- Clang (C/C++), Rust, Swift, and Julia all compile through LLVM IR

Java: compile once, run anywhere



- `javac` compiles Java to **bytecode** (`.class`), not to machine code
- Each platform has its own **Java Virtual Machine (JVM)**
- The JVM interprets the bytecode and **JIT-compiles** hot code to machine code
- Same `.jar` runs on an Intel laptop and an ARM phone

Dcoetzee. Java virtual machine architecture. Wikimedia Commons, CC0.

C vs. Python: what happens when you run a program?

C: compile, then execute

```
cc hello.c -o hello   # compile
./hello              # execute
```

```
hello.c
   ↓ C compiler
native machine code (hello)
   ↓
CPU executes the program
```

Python: one command

```
python3 hello.py      # compile + run
```

```
hello.py
   ↓ CPython parses + compiles
Python bytecode (in memory)
   ↓ CPython VM executes bytecode
CPU executes interpreter code
```

- **Python still compiles**: `python3` handles compilation and execution for you
- Edit the source: rebuild the C executable; rerun `python3 hello.py` for Python

Python documentation: https://docs.python.org/3/glossary.html · CPython interpreter: https://github.com/python/cpython/blob/main/InternalDocs/interpreter.md

Python's interpreter: advantages and disadvantages

Advantages

- **Quick iteration**: edit and rerun with one command; experiment in a REPL
- **Portable source**: a compatible interpreter handles each platform
- **Flexible**: dynamic types and automatic memory management

Disadvantages

- **Slower Python loops**: bytecode dispatch and object handling add work
- **More memory**: object metadata and pointers add overhead
- **Runtime errors**: type mistakes may surface only on an executed path

Example: `a + b` — C can compile integer addition to a CPU instruction; Python selects an operation based on the objects' types at runtime.

Imperative vs. declarative

Imperative: say **how**

```
total, count = 0, 0
for o in orders:
    if o.customer_id == 42:
        total += o.price
        count += 1
avg = total / count
```

Declarative: say **what**

```
SELECT AVG(price)
FROM orders
WHERE customer_id = 42;
```

```
cheap = sorted(filter(lambda o: o.price < 10, orders),
               key=lambda o: o.price)
```

- SQL never says *which* rows to visit or *in what order*; the database decides

Declarative languages: advantages and limitations

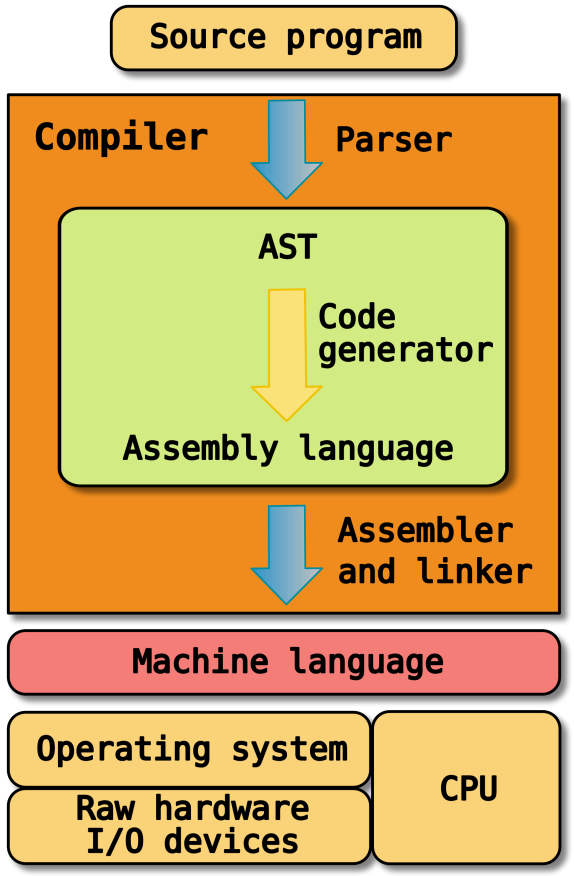
Advantages

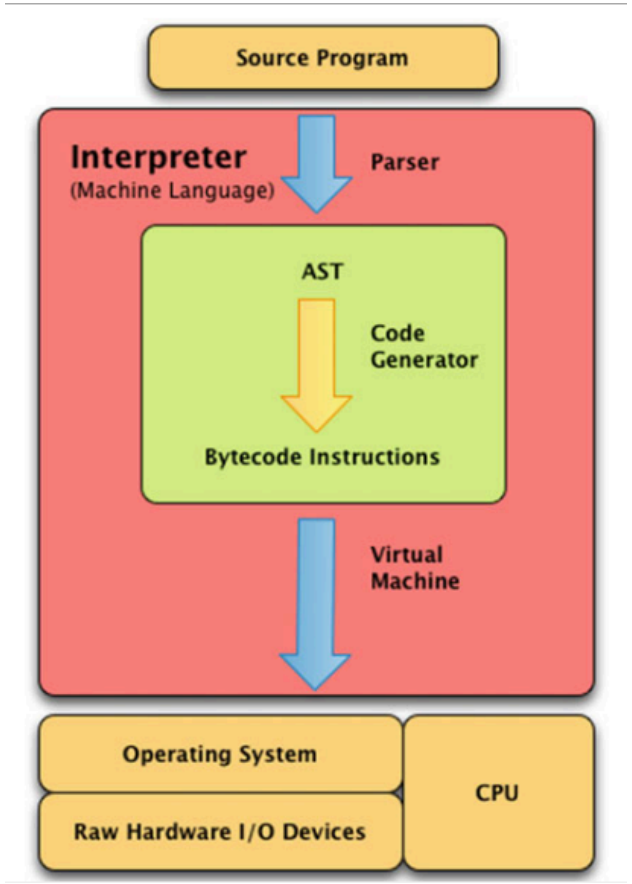
- **Short**: one line instead of a loop
- **The system picks the plan**: indexes, join order, parallel workers
- Same query gets faster when the system improves
- Easy to run on many machines (Spark SQL, BigQuery)

Limitations

- Hard to express **arbitrary logic** (custom loops, state)
- Speed depends on the **optimizer**: a bad plan can be 1000× slower
- Harder to **debug**: the code you wrote is not the code that runs
- Python lambdas still run in the interpreter, one call per element

Compiled vs. interpreted





Compiled (C, C++, Rust, Go)

Interpreted (Python, JavaScript)

Seda Kh. Compiler scheme (en); Code interpreter scheme. Wikimedia Commons, CC BY-SA 4.0.

Python's copyij, as the interpreter sees it

`dis.dis(copyij)`, Python 3.12, inner loop:

```
FOR_ITER        (to 100)
STORE_FAST      j
LOAD_FAST       src
LOAD_FAST       i
LOAD_FAST       n
BINARY_OP       *
LOAD_FAST       j
BINARY_OP       +
BINARY_SUBSCR
LOAD_FAST       dst
LOAD_FAST       i
LOAD_FAST       n
BINARY_OP       *
LOAD_FAST       j
BINARY_OP       +
STORE_SUBSCR
JUMP_BACKWARD   (to 52)
```

- **17 bytecode instructions** per element
- Each one is a trip through the interpreter's C loop: fetch, dispatch, check types, update reference counts
- `i * n + j` creates **new int objects** every time
- Result: **~55 ns ≈ 250 cycles per element**
- C `-O2` does the same work in **< 1 cycle**

Compiled vs. interpreted: trade-offs

	Compiled (C, C++, Rust, Go)	Interpreted (Python, JavaScript)
Speed	**fast**: machine code, optimized ahead of time	slow: interpreter work on every operation
Start-up	compile step first (seconds to minutes)	**runs immediately**; great for notebooks
Portability	rebuild for each CPU and OS	**same source runs** wherever the interpreter runs
Types	checked at compile time	checked at run time; more flexible
Errors	many caught before running	found when that line runs
Typical use	OS, databases, game engines, NumPy itself	glue code, data analysis, web pages

- Python's trick: write the **slow-to-write, fast-to-run** parts in C, and the rest in Python

The middle ground: just-in-time (JIT) compilation

- Start by interpreting; find **hot** code; compile it to machine code **while running**
- **JavaScript**: V8 (Chrome, Node.js) JIT-compiles hot functions
- **Java**: the HotSpot JVM JIT-compiles hot bytecode
- **Python**: PyPy is a JIT-based Python; CPython 3.13 added an experimental JIT
- **Numba / JAX / PyTorch** `compile`: JIT for numerical Python code

Inside NumPy: one block of typed memory

Python list: an array of pointers, each to a separate 28-byte int object



NumPy array: one contiguous buffer of raw 8-byte values, plus dtype / shape / strides



2048 × 2048 integers	**Memory**
Python list of ints	32 MB of pointers + 112 MB of int objects = **144 MB**
NumPy `int64` array	**32 MB** (`strides = (16384, 8)`)

Inside NumPy: the loop runs in C

```
dst[i, :] = src[i, :]     # one Python call → one C loop over 2,048 values
dst[:] = src              # one Python call → one C loop over 4.2 M values
```

2048 × 2048 int64 copy (M4 Pro)	Time	Per element
Pure Python loop	231 ms	55 ns
NumPy, 2,048 row copies	0.77 ms	0.18 ns
NumPy, one call `dst[:] = src`	**0.45 ms**	**0.11 ns**

- Python overhead: **~116 ns per NumPy call**, paid **once per call**, not per element
- 2,048 row calls × 116 ns ≈ 0.24 ms: most of the gap between the two NumPy rows

What an optimizing compiler does: copyij

`-O0` : inner loop, **19 instructions per element**

```
ldr  x8, [sp]          ; load j from memory
subs x8, x8, #2048
b.ge LBB0_6
ldr  x8, [sp, #24]     ; load src pointer
ldr  x9, [sp, #8]      ; load i
add  x8, x8, x9, lsl #14
ldr  x9, [sp]          ; load j again
ldr  x8, [x8, x9, lsl #3]
...                    ; same again for dst
str  x8, [x9, x10, lsl #3]
ldr  x8, [sp]          ; j = j + 1
add  x8, x8, #1
str  x8, [sp]
```

`-O2` : **6 instructions per 8 elements**

```
ldp  q0, q1, [x12, #-32]
ldp  q2, q3, [x12], #64
stp  q0, q1, [x13, #-32]
stp  q2, q3, [x13], #64
subs x14, x14, #8
b.ne LBB0_6
```

- Keeps `i`, `j`, pointers in **registers**
- Replaces `i * 2048 + j` with a moving pointer
- **Vectorizes**: 64 bytes per step (SIMD)

Compiler optimization: tail recursion

```c
long gcd(long a, long b) {
    if (b == 0) return a;
    return gcd(b, a % b);   // tail call
}
```

`clang -O2` : the call becomes a **loop**

```
LBB0_1:
  mov  x8, x1
  sdiv x9, x0, x1
  msub x1, x9, x1, x0
  mov  x0, x8
  cbnz x1, LBB0_1      ; jump back, no call
```

`sum_to(n, acc)` = `sum_to(n - 1, acc + n)` , with n = 100,000,000:

- C `-O0` : **crashes** (segfault: 100 M stack frames)
- C `-O2` : no loop at all; clang replaced it with **n(n+1)/2**
- Python: `RecursionError` at depth **1,000**; CPython does not optimize tail calls

Query optimization: same SQL, different plan

```sql
SELECT AVG(price) FROM orders WHERE customer_id = 42;   -- 2 M rows
```

	Plan chosen by SQLite (`EXPLAIN QUERY PLAN`)	**Time**
No index	`SCAN orders` (reads all 2 M rows)	37 ms
After `CREATE INDEX ... ON orders(customer_id)`	`SEARCH orders USING INDEX idx_customer`	**0.007 ms**

- **5,000× faster**, and the query did not change
- The optimizer also chooses **join order**, pushes filters down, and picks algorithms
- This is the payoff of a declarative language: the system can rewrite *how*

Why is C faster than Python?

- Python runs **17 bytecodes per element**, each a trip through the interpreter
- Every value is a **heap object** with a type and reference count; types are checked at run time
- C is compiled **ahead of time** to machine code that loads and stores raw 8-byte values
- The C compiler can also **optimize**: registers, pointer arithmetic, SIMD

copyij, 2048 × 2048	Time	≈ cycles per element
Pure Python	231 ms	~250
C `-O0`	4.7 ms	~5
C `-O2`	0.6 ms	~0.6

Why is NumPy faster, even than unoptimized C?

- **vs. pure Python**: one contiguous typed array + one C loop per call, so the interpreter runs **once per call**, not once per element
- **vs. C `-O0`**: NumPy's loops are C code compiled **with optimization** (and a tuned `memcpy` for contiguous copies); our `-O0` C keeps every variable in memory
- **vs. C `-O2`**: about the same; a single `dst[:] = src` call (0.45 ms) even edges out our hand-written loop (0.6 ms)
- Lesson: **use the library**, and make sure its loop does the work, not yours

Q2: Why is C faster than Python? Why is NumPy faster?

- A **compiler** translates high-level code → IR → machine code, and optimizes on the way
- **Interpreted** languages trade speed for flexibility; **JIT** compilers win some of it back
- **Declarative** languages (SQL) let the system choose a fast plan
- **NumPy** gets C speed by keeping data in one typed array and the loop in compiled C
- Speed depends on **where the work happens**: interpreter, compiler, library, or optimizer

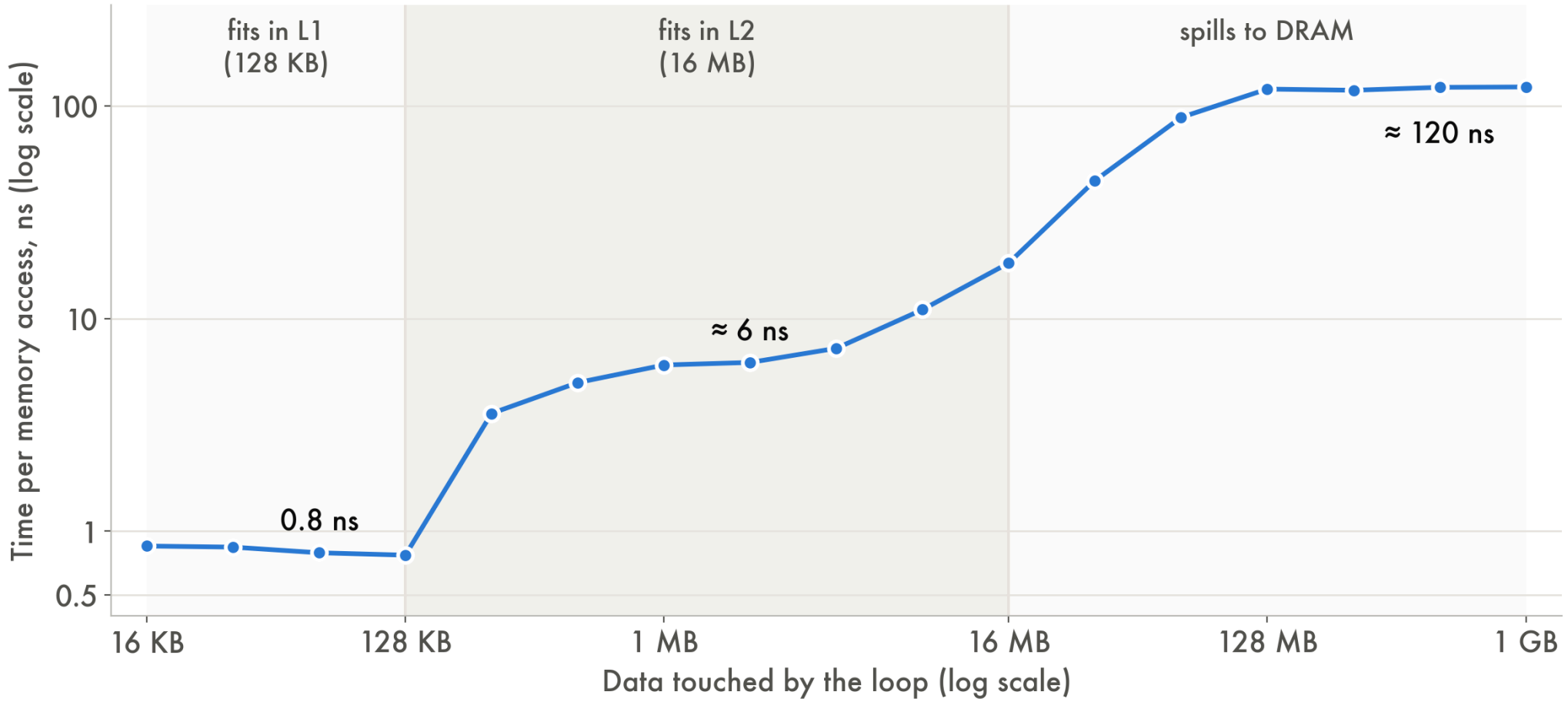
Q3: Why is copyij faster than copyji?

Same instructions, very different time

2048 × 2048 copy, C `-O2`	**Time**	**≈ cycles per copy**
copyij (row by row)	0.6 ms	0.6
copyji (column by column)	38.9 ms	**42**

- Same number of loads and stores, same compiler, same machine
- copyji spends almost all of its time **waiting for memory**
- To see why, we need to know **how slow memory is**, and **what the hardware does about it**

The problem: memory is slow



- Same load instruction: **0.8 ns** from L1, **~6 ns** from L2, **~120 ns** from DRAM

The problem keeps getting worse

- **1985**: one DRAM access ≈ **one** CPU cycle
- **Today**: one DRAM access (~120 ns) ≈ **500** cycles at 4.5 GHz
- CPU speed grew for decades; DRAM **latency** barely moved
- This growing **CPU–memory gap** is often called the **memory wall**
- A CPU that must wait for DRAM on every load runs at **< 1%** of its speed

Bryant, O'Hallaron. Computer Systems: A Programmer's Perspective, 3rd ed., Ch. 6 (CPU–memory gap).

Why not make all memory fast?

Technology	Where	Latency	Typical size	Cost
SRAM (registers, caches)	on the CPU chip, next to the core	~1 ns	KB – tens of MB	very high; uses chip area
DRAM (main memory)	separate chips on the board	~100 ns	16 GB – 2 TB	~$15 / GB
Flash SSD	storage	~100 µs	1 – 30 TB	~$0.10 / GB
Hard disk	storage	~10 ms	10 – 30 TB	~$0.02 / GB

- Fast memory is **small, expensive, and power-hungry**, and must sit **close to the core**
- Large memory is cheap but **far away**: signals take time to travel and to be found
- Physics forces a trade-off: **fast, big, cheap: pick two**

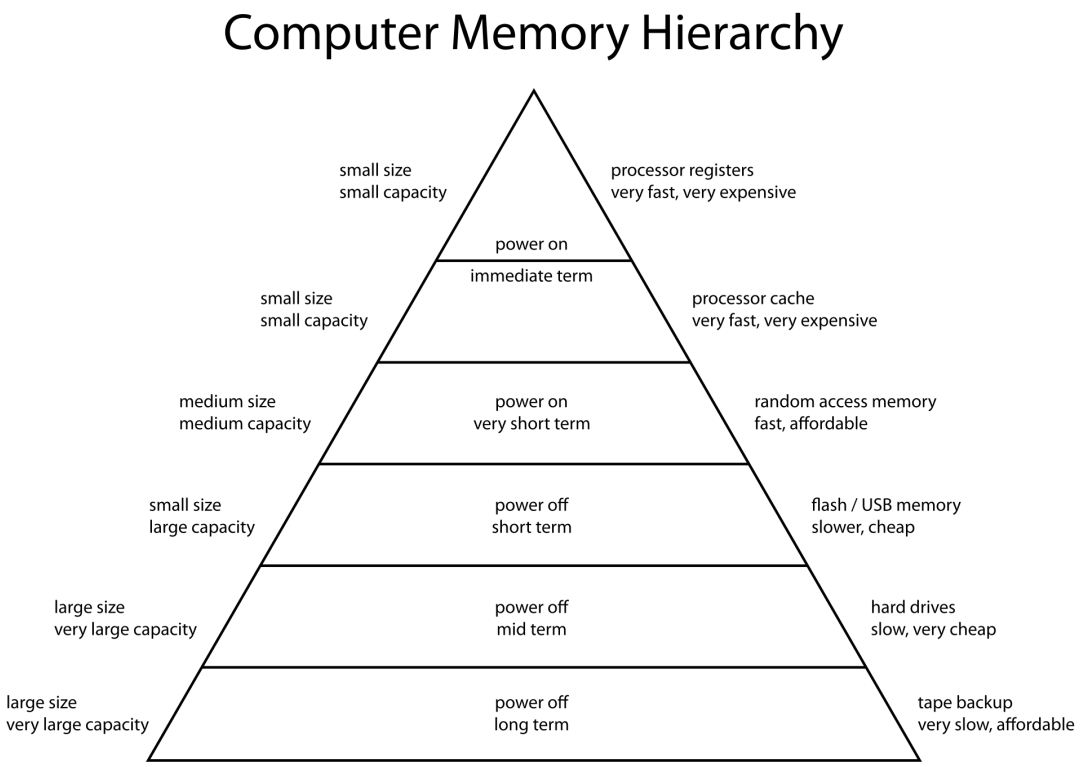
And disk is slower still



- Spinning platters + a moving arm
- One access = **seek** (~9 ms) + **rotation** (~4 ms at 7,200 RPM) + transfer (~0.02 ms)
- **≈ 13 ms**: about **100,000×** a DRAM access
- The first byte is expensive; the rest of the sector is nearly free

Image courtesy of Seagate Technology. Timing example after Bryant, O'Hallaron, CS:APP Ch. 6.

The solution: a memory hierarchy

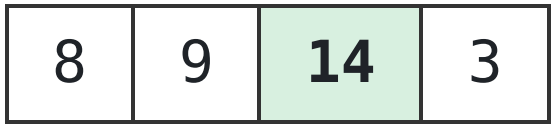


- Stack the technologies: **small and fast on top**, **large and cheap below**
- Each level is a **cache** for the level below it
- Keep the data you are using **near the top**
- Registers ← L1 ← L2 ← DRAM ← SSD ← remote storage

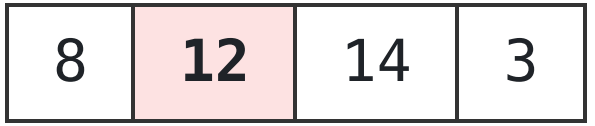
Danlash. Computer Memory Hierarchy. Wikimedia Commons, public domain.

How a cache works: hit and miss

Request block 14: it is in the cache → **hit**, served in ~1 ns



Request block 12: not in the cache → **miss**

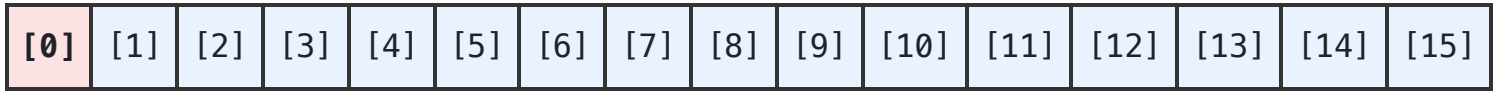


- Fetch block 12 from the level below (slow)
- **Placement**: where block 12 goes
- **Replacement**: which block is evicted (here, 9)

- Average time ≈ hit time + **miss rate** × miss penalty: a small miss rate matters a lot

Memory moves in blocks: cache lines

- A miss never fetches just one value; it fetches the whole **cache line** around it
- On the M4 Pro, a cache line is **128 bytes** = **16** `long`s (`sysctl hw.cachelinesize`)



- Ask for `[0]` (miss) → `[1]` … `[15]` arrive **for free**
- Using all 16 values: **1 miss per 16 accesses**
- Using only one: **1 miss per access**

Prefetching: fetch before you are asked

- The hardware watches the stream of addresses
- If it sees a pattern (next line, next line, ...), it **fetches ahead**
- By the time the program asks, the data is already in the cache
- Works best for **sequential** access; random jumps defeat it
- Software does the same: OS read-ahead for files, video buffering

Caching is everywhere

Cache	What is cached	Where	Latency (cycles)	Managed by
Registers	8-byte words	CPU core	0	compiler
L1 / L2 cache	64–128-byte lines	on chip	4 – 30	hardware
Virtual memory	4–16 KB pages	DRAM	~500	hardware + OS
Page cache	parts of files	DRAM	~500	OS
Disk cache	disk sectors	disk controller	~100,000	disk firmware
Browser cache	web pages	local disk	~10,000,000	browser
CDN / web cache	web pages	remote servers	~1,000,000,000	proxy servers

Why it works: locality

- **Principle of locality**: programs tend to use data and instructions with addresses **near or equal to** those they used recently
- **Temporal locality**: a recently used item is likely to be **used again** soon
  - keep it in the cache
- **Spatial locality**: items **near** a recently used item are likely to be used soon
  - fetch the whole cache line; prefetch the next one
- Caches are a **bet on locality**; good programs make the bet pay off

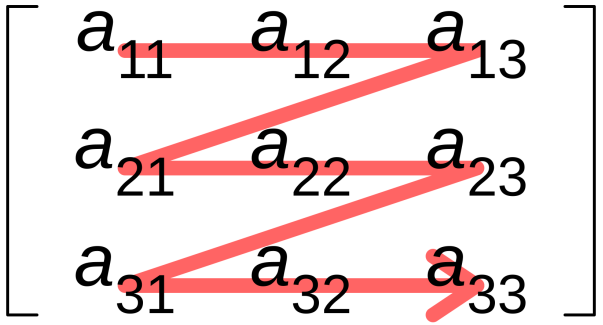
Locality in a simple loop

```
total = 0
for x in nums:          # nums = [1, 2, 3, 4, 5, 7]
    total += x
```

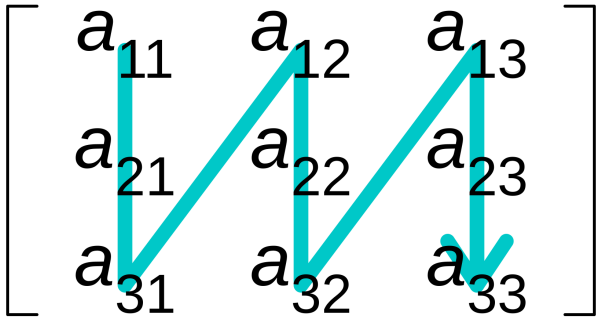
- **Data**
  - `nums` elements, one after another (stride 1) → **spatial**
  - `total` on every iteration → **temporal**
- **Instructions**
  - executed in sequence → **spatial**
  - the loop body repeats → **temporal**

How C lays out a 2D array

Row-major order



Column-major order



- C (and NumPy by default) store arrays **row by row**: **row-major**
- `&a[i][j]` = base + (i × 2048 + j) × 8 bytes
- `a[i][j]` → `a[i][j+1]`: **8 bytes** apart
- `a[i][j]` → `a[i+1][j]`: **16 KB** apart
- Fortran, MATLAB, R: **column-major**

Cmglee. Row and column major order. Wikimedia Commons, CC BY-SA 4.0.

copyij vs. copyji, one cache line at a time

copyij (stride 1)

- Walks along a row: 8 bytes per step
- Each 128-byte line: **1 miss, then 15 hits**
- Sequential: the **prefetcher** fetches the next line early
- Compiler adds **SIMD**: 64 bytes per step

copyji (stride 2048)

- Walks down a column: **16 KB** per step
- Every access touches a **new line**: uses 8 of its 128 bytes
- One column pass touches 2 × 2048 lines = **512 KB**, more than the 128 KB L1
- By the next column, those lines are **evicted**: **every access misses L1**

Evidence: it is the cache

NumPy, row-major, time per element (`copy_demo.py --part 4`)

N	Matrix size	copyij	copyji	copyji / copyij
64	32 KB	2.51 ns	2.69 ns	1.1×
256	512 KB	0.74 ns	3.10 ns	4.2×
1024	8 MB	0.29 ns	7.22 ns	25×
2048	32 MB	0.19 ns	9.60 ns	**52×**
4096	128 MB	0.17 ns	9.38 ns	**54×**

- While the matrix **fits in cache**, the loop order barely matters
- Once it does **not**, column order pays a miss on every access

Three kinds of cache misses

- **Cold (compulsory)**: the first access to a block; the cache started empty
  - every program pays these once
- **Capacity**: the data in use (the **working set**) is bigger than the cache
  - **copyji**: one column pass needs 512 KB of lines; L1 holds 128 KB
- **Conflict**: the cache has room, but several blocks map to the **same slot**
  - e.g., blocks 0, 8, 0, 8, … in a cache where block *i* must go to slot *i* mod 8

Same lesson, bigger code: matrix multiplication

ijk: B is read down a column

```
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j];
```

875 ms

ikj: B and C are read along rows

```
for (i = 0; i < N; i++)
  for (k = 0; k < N; k++)
    for (j = 0; j < N; j++)
      C[i][j] += A[i][k] * B[k][j];
```

122 ms (7.2× faster)

- Same math, same result; 1024 × 1024 doubles, `-O2`, M4 Pro
- Stride-1 inner loop → cache hits (and lets the compiler use SIMD)
- Libraries like BLAS (inside NumPy) go further: they **tile** the matrices to fit in cache

Exam-style question

Q: Permute the loops so the function scans `a` with a stride-1 pattern.

```
int sum_array_3d(int a[M][N][N]) {
    int i, j, k, sum = 0;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < M; k++)
                sum += a[k][i][j];
    return sum;
}
```

Exam-style question: answer

Make the last index (`j`) the innermost loop:

```
for (k = 0; k < M; k++)
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            sum += a[k][i][j];
```

- Row-major: the **last** index varies fastest in memory
- Loop order should match the index order: `k`, then `i`, then `j`

Why the hierarchy works

- Fast storage costs more per byte; slow storage is cheap
- Thanks to **locality**, most accesses hit the small fast levels
- So level *k* + *1* can be slower, and therefore **larger and cheaper**
- **Big idea**: the hierarchy gives you storage that costs about as much as the **cheap** level but runs at about the speed of the **fast** level
- It only works when the program **has locality**; copyji breaks the deal

The same idea beyond the CPU

- **pandas vs. Dask**: pandas needs the whole table in DRAM; Dask keeps it on disk and brings chunks into DRAM
- **Browser cache / CDN**: keep popular pages close to the user
- **Databases**: a buffer pool keeps hot pages in DRAM
- **Language models**: generating each token reads all the weights (~350 GB for a large model)
  - Weights do not fit in cache; memory **bandwidth** decides speed
  - Open question: how would you organize the hierarchy?

Q3: Why is copyij faster than copyji?

- **Problem**: DRAM is ~150× slower than L1, and the gap keeps growing
- **Solution**: a memory hierarchy of caches, moving data in **cache lines**, with **prefetching**
- **Why it works**: **locality**: programs reuse recent data and nearby data
- copyij walks memory **in order**: 1 miss per 16 accesses, prefetched
- copyji jumps **16 KB** each step: a miss on every access
- Speed depends on the **memory access pattern**, not just the instruction count

A memory hierarchy in the cloud: Amazon S3

S3 storage class	$ per GB-month	1 TB for a month	Time to first byte	Min. stay	Retrieval fee
Standard	$0.023	$23.00	milliseconds	none	none
Standard-Infrequent Access	$0.0125	$12.50	milliseconds	30 days	$0.01 / GB
Glacier Instant Retrieval	$0.004	$4.00	milliseconds	90 days	$0.03 / GB
Glacier Flexible Retrieval	$0.0036	$3.60	minutes to hours	90 days	$0 – 0.03 / GB
Glacier Deep Archive	**$0.00099**	**$0.99**	**up to 12 – 48 hours**	180 days	$0.0025 – 0.02 / GB

- Same pattern as registers → caches → DRAM → disk: **faster = more expensive per byte**
- S3 Standard costs **23×** more per byte than Deep Archive

AWS S3 pricing, US East (N. Virginia), checked October 2026 via the AWS Price List API; aws.amazon.com/s3/storage-classes.

Why is S3 Standard 23× more expensive than Glacier?

- **You pay for being ready**: Standard keeps data on always-on servers, readable in milliseconds, as often as you like
- **Glacier can be slow, so it can be cheap**: AWS can use denser, cheaper, mostly idle storage and serve requests in batches over hours (AWS does not disclose the media)
- **The cost moves from storing to reading**: retrieval fees, minimum stay, and hours of waiting are Glacier's **miss penalty**
- **Lifecycle rules move cold data down the hierarchy**: Standard → Infrequent Access after 30 days → Deep Archive after 180 days, the same bet on **locality** as a CPU cache
- **Example**: 100 TB of logs for a year: **$27,600** in Standard vs. **$1,188** in Deep Archive; reading it all back once adds ~$250 (bulk) and **up to 48 hours**

Q4: How can we make copyij even faster?

Where does the time go now?

- copyij in NumPy: **32 MB in, 32 MB out, 0.9 ms** → about **70 GB/s**
- The access pattern is fixed: every cache line is used in full and prefetched
- The CPU is no longer waiting for a miss; it is waiting for **bandwidth**: this laptop's memory delivers ~125 GB/s to one core (measured below)
- Four levers are left:
  i. **Move fewer bytes**: store each number in fewer bytes
  ii. **Do more per instruction**: SIMD, one instruction copies 2, 4, or 8 values (we saw `-O2` do this in Q2)
  iii. **Use more workers**: several cores, or a GPU, each copying a slice
  iv. **Do not copy**: a view or a pointer swap moves 0 bytes
- Today: **lever 1**, because it decides how numbers are stored, and it is why GPUs got so fast at machine learning

Experiment: same copy, smaller numbers

8192×8192 array, `np.copyto`, Apple M4 Pro, best of 9 runs

Number format	Bytes per number	Array size	Time	Speedup	Bandwidth
float64	8	537 MB	8.7 ms	1×	123 GB/s
float32	4	268 MB	4.3 ms	**2.0×**	125 GB/s
float16	2	134 MB	2.1 ms	**4.2×**	129 GB/s
int8	1	67 MB	1.0 ms	**8.4×**	130 GB/s

- The bandwidth is the same every time: the memory system is the bottleneck
- So the time is proportional to the **bytes**, not to the number of numbers
- 8× fewer bytes per number → 8× faster. What did we give up?

To answer that, we need to know what the bits mean

- An integer is easy: each bit is a power of 2
  - $13 = 8 + 4 + 1 = 1101_2$
- A real number has a **binary point**: bits to its right are 1/2, 1/4, 1/8, …
  - $0.625 = 1/2 + 1/8 = 0.101_2$
- **Fixed point**: put the binary point at a fixed position
  - 32 bits as 16.16: largest ≈ 65,536, smallest step 1/65,536
  - Cannot hold 10^{-10} and 10^{13} in the same format

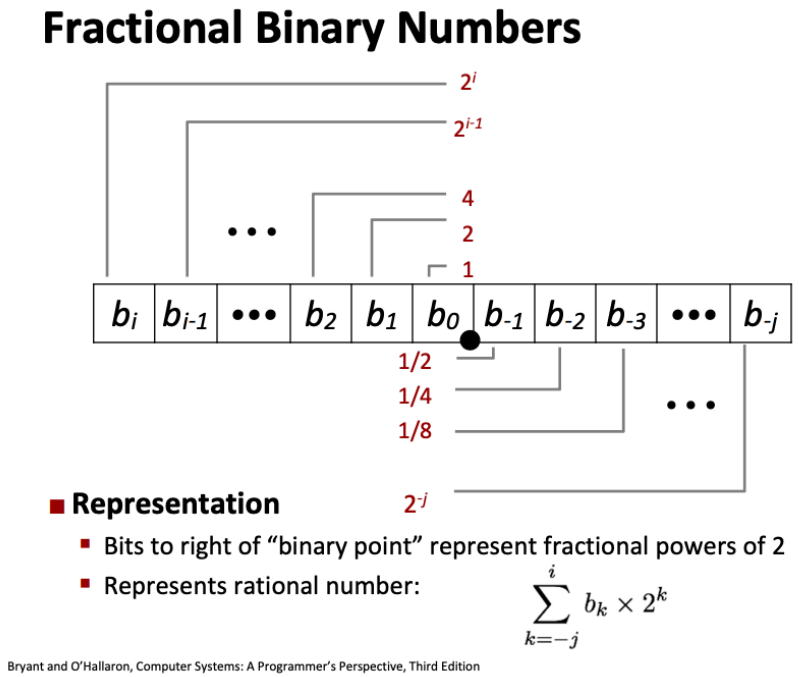
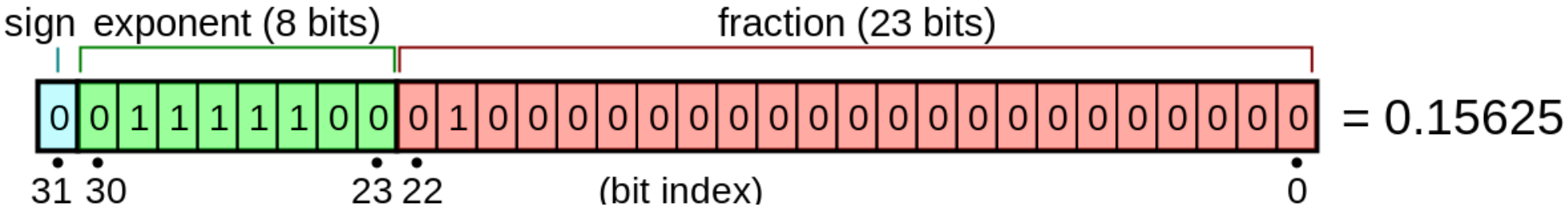


Figure: Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, 3rd ed.

Floating point: scientific notation in binary

- **Idea 1**: write the number in scientific notation, so the scale and the digits are separate
  - $0.000000000123455 = \mathbf{1.23455} \times 10^{\mathbf{-10}}$
  - $99{,}999{,}999{,}999{,}999 = \mathbf{9.999999999999} \times 10^{\mathbf{13}}$
- **Idea 2**: store the two parts in separate fields: the **exponent** and the **fraction** (the significant digits)
- **Idea 3**: everything is binary, so the base is 2, and the number is $\mathbf{\pm 1.fff..._2 \times 2^e}$
  - $0.625 = 0.101_2 = \mathbf{1.01_2 \times 2^{-1}}$
  - The leading digit of a normalized binary number is always 1, so it is not stored: a free bit
- The binary point "floats" to wherever the exponent says, hence **floating point**

IEEE 754 single precision (FP32): 1 + 8 + 23 bits

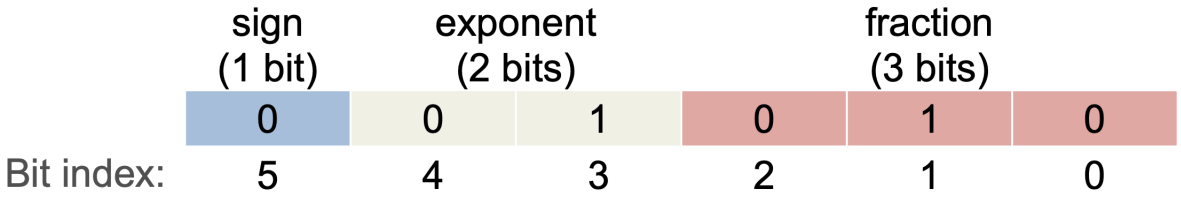


$$(-1)^{sign} \times 2^{exponent-127} \times (1 + \sum_{i=1}^{23} b_{23-i} 2^{-i})$$

- **Sign** (1 bit), **exponent** (8 bits, stored with a bias of 127), **fraction** (23 bits after the implicit leading 1)
- Example above: exponent field 124 → $2^{124-127}$ = 1/8; fraction 0.01_2 = 1/4; value = (1/8) × (1 + 1/4) = **0.15625**
- The same recipe, with different field widths, gives every format we will see today

Figure: Wikipedia, "Single-precision floating-point format" (CC BY-SA). IEEE 754-2019.

Let's design a 6-bit float: 1 + 2 + 3 bits



$$(-1)^{sign} \times 2^{exponent-2} \times (1 + \sum_{i=1}^{3} b_{3-i} 2^{-i})$$

Encode 0.625:

- $0.625 = 0.101_2 = 1.01_2 \times 2^{-1}$
- Sign: + → **0**
- Exponent: −1 + bias 2 = 1 → **01**
- Fraction: the bits after "1.": **010**
- Result: **0 01 010**

Check: $(-1)^0 \times 2^{1-2} \times (1 + 0/2 + 1/4 + 0/8) = 0.5 \times 1.25 = 0.625$ ✓

All 32 positive values of our 6-bit float

Exponent	Scale 2^{e-2}	Values ($1.fff_2$ × scale)	Step
00	1/4	0.25, 0.281, 0.313, 0.344, 0.375, 0.406, 0.438, 0.469	1/32
01	1/2	0.5, 0.563, 0.625, 0.688, 0.75, 0.813, 0.875, 0.938	1/16
10	1	1, 1.125, 1.25, 1.375, 1.5, 1.625, 1.75, 1.875	1/8
11	2	2, 2.25, 2.5, 2.75, 3, 3.25, 3.5, 3.75	1/4

- The values are **not evenly spaced**: each time the number doubles, the gap doubles too
- Relative precision is constant: about 1/8, so **three significant bits** (≈ 1 decimal digit)
- **Exponent bits set the range** (here 0.25 to 3.75); **fraction bits set the precision** (the step within each range)
- 2.4 is not representable: it is stored as 2.5. Every float format is a trade-off between range and precision

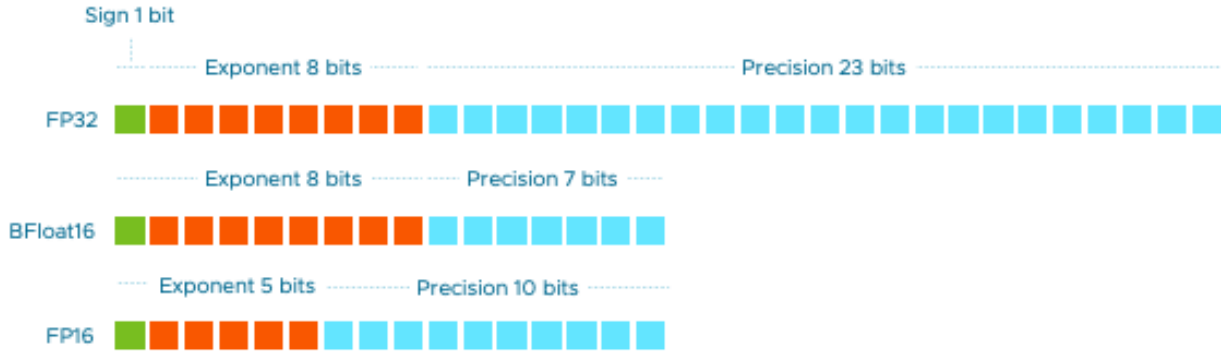
The float family: from FP64 to FP4

Format	Bits	Sign	Exponent	Fraction	Largest value	Relative precision	Who uses it
FP64 (double)	64	1	11	52	1.8×10^{308}	$2^{-52} \approx 10^{-16}$ (16 digits)	Python `float`, NumPy default
FP32 (single)	32	1	8	23	3.4×10^{38}	$2^{-23} \approx 10^{-7}$ (7 digits)	C/Java `float`, graphics
TF32	19	1	8	10	3.4×10^{38}	$2^{-10} \approx 10^{-3}$ (3 digits)	NVIDIA Tensor Cores, stored in 32 bits
FP16 (half)	16	1	5	10	65,504	$2^{-10} \approx 10^{-3}$ (3 digits)	mixed-precision training (2017)
BF16 (bfloat16)	16	1	8	7	3.4×10^{38}	$2^{-7} \approx 10^{-2}$ (2 digits)	LLM training and weights (2018)
FP8 E4M3	8	1	4	3	448	$2^{-3} = 1/8$	H100 Tensor Cores, DeepSeek-V3
FP8 E5M2	8	1	5	2	57,344	$2^{-2} = 1/4$	gradients in FP8 training
FP4 E2M1	4	1	2	1	6	$2^{-1} = 1/2$	Blackwell Tensor Cores, quantized LLMs

- Same recipe every time: sign, exponent (range), fraction (precision); only the widths change
- Our 6-bit float sits between FP8 and FP4. Our copy ran float64 → float32 → float16: 16 digits → 7 → 3

IEEE 754-2019; NVIDIA, "FP8 Formats for Deep Learning" (2022); OCP Microscaling Formats specification v1.0 (2023).

BF16 vs. FP16: where do you spend 16 bits?



- **FP16**: 5 exponent bits, 10 fraction bits: 3 digits of precision, but the largest value is **65,504** and the smallest normal is 6×10^{-5}
- **BF16**: 8 exponent bits, 7 fraction bits: only 2 digits, but the **same range as FP32**
- BF16 is just FP32 with the low 16 bits cut off: conversion is **truncation**

- Deep learning chose **range over precision**: gradients and activations span many orders of magnitude; a few extra digits do not change the trained model much
- FP16 training needs **loss scaling** to keep gradients from underflowing to 0 or overflowing to ∞; BF16 does not

Figure: Google Cloud, "BFloat16: The secret to high performance on Cloud TPUs" (2019).

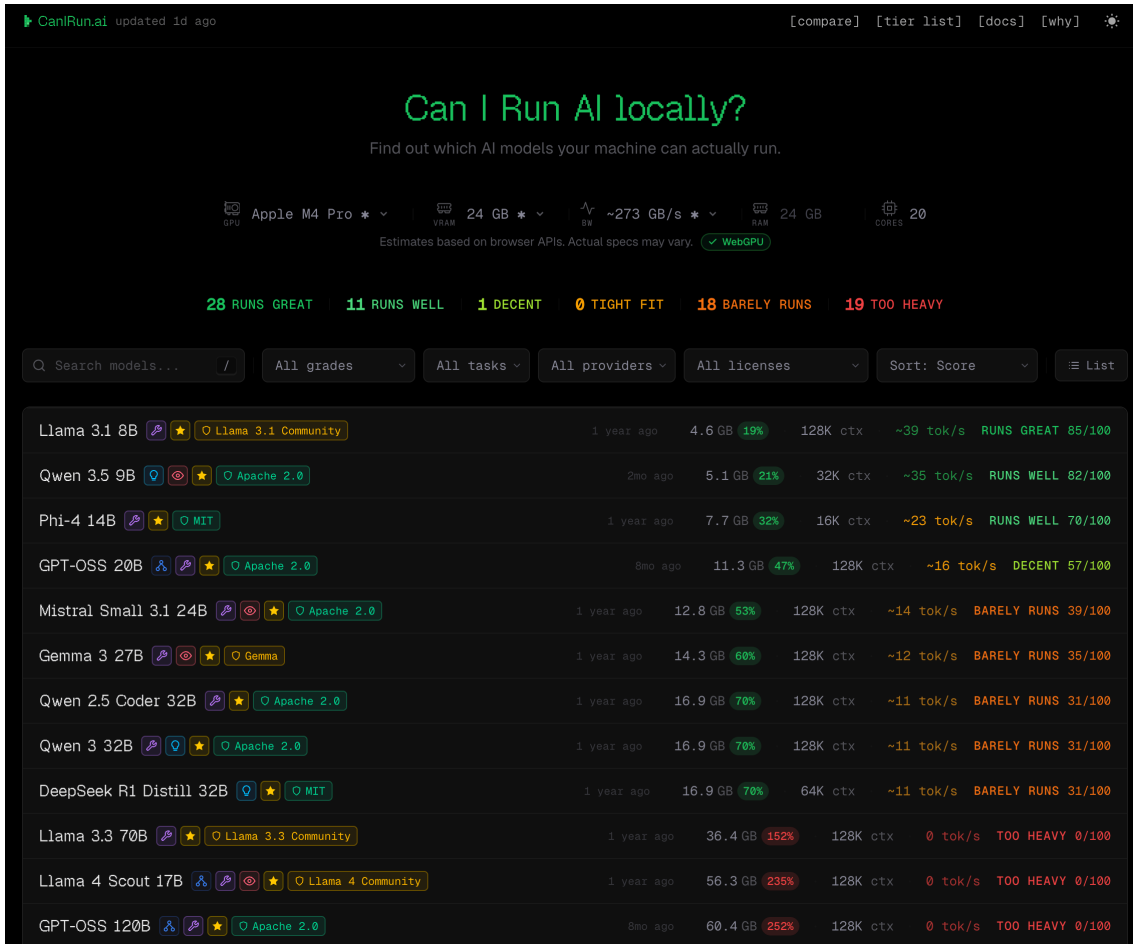
FP8 and FP4: how can 16 values represent a model?

- **FP4 E2M1** has 16 bit patterns: **±{0, 0.5, 1, 1.5, 2, 3, 4, 6}**. That is all
- Trick: **block scaling**. Store one shared scale per small block of numbers, and 4-bit values relative to it
  - **MXFP4** (Open Compute, 2023): 32 values share one 8-bit power-of-two scale → 4.25 bits per value
  - **NVFP4** (NVIDIA Blackwell, 2025): 16 values share one FP8 scale, plus one FP32 scale per tensor → 4.5 bits per value
- A float with a shared exponent is just a **fixed-point block**: the exponent bits are paid once per 16 or 32 numbers, not once per number
- Weights within a block are similar in magnitude, so the block scale does most of the work; the 4 bits only pick among 8 magnitudes

Example: gpt-oss-120b (OpenAI, 2025) is released in MXFP4: 117 B parameters → **~60 GB**, so it fits on **one 80 GB H100**. In BF16 it would be **234 GB**, i.e., three GPUs plus the traffic between them

OCP Microscaling Formats (MX) Specification v1.0, 2023; NVIDIA Developer Blog, "Introducing NVFP4", June 2025; OpenAI gpt-oss model card, August 2025.

Can I run it on my laptop? Bytes per weight decide



- Same laptop as this lecture: M4 Pro, 24 GB of memory
- The site lists each model at **4-bit** weights: Llama 3.1 8B = 4.6 GB, Llama 3.3 70B = 36 GB
- 8 B parameters × 0.5 bytes ≈ 4 GB (plus a small overhead); in BF16 it would be **16 GB**, in FP32 **32 GB**
- Memory size is a hard wall; the **tok/s** column is the memory **bandwidth** wall: every token reads every weight

canirun.ai, "Can I Run AI locally?", checked October 2026 on this laptop.

GPU speed depends on the number format

Peak throughput, dense (no sparsity), TFLOPS

Format	H100 SXM (2022)	Blackwell, per GPU in GB200 (2025)	vs. FP32
FP64	34 (67 Tensor Core)	40	0.5×
FP32	67	80	1×
TF32 Tensor Core	495	1,250	15×
BF16 / FP16 Tensor Core	989	2,500	31×
FP8 Tensor Core	1,979	5,000	62×
FP4 Tensor Core	not supported	**10,000**	**125×**

- Each halving of the bits roughly **doubles** the Tensor Core rate; FP32 → FP4 is **125×** on Blackwell
- FP32 and FP64 barely moved in three years: the transistors went to **low-precision matrix units**
- Why: a 4-bit multiplier is a tiny fraction of a 32-bit one, so far more fit on a chip, and each operand is 8× fewer bytes

NVIDIA H100 and GB200 NVL72 product pages, October 2026. NVIDIA headlines "with sparsity" numbers, which are 2× these; GB200 per GPU = NVL72 rack ÷ 72.

Why fewer bits make everything faster

- **Bandwidth**: half the bytes per number, half the time when memory is the bottleneck
  - Our copy: 8.7 ms → 1.0 ms. LLM token generation: every token reads all the weights, so bytes per weight sets tokens per second
- **Compute**: smaller multipliers → more of them per chip → Tensor Core throughput doubles per halving
- **Capacity**: the data fits one level higher in the **memory hierarchy**
  - 4× more numbers per cache line, per GB of GPU memory, per GPU
  - gpt-oss-120b: 234 GB in BF16 (3 GPUs + NVLink traffic) vs. 60 GB in MXFP4 (1 GPU, no traffic)
- **Cost**: fewer GPU-hours per training run, fewer GPUs per served model

This is the same lesson as Q3, one level up: **the bottleneck is moving data, so move fewer bytes**

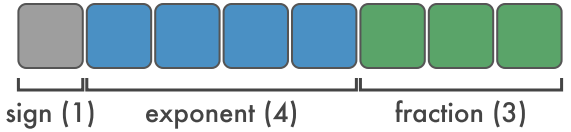
What do we give up? Precision

```
>>> 0.1 + 0.2                  # even in FP64
0.30000000000000004
>>> (0.1 + 0.2) + 0.7
1.0
>>> 0.1 + (0.2 + 0.7)          # not associative
0.9999999999999999
>>> np.float16(70000)          # beyond 65,504
inf
>>> np.float16(0.1) + np.float16(0.2)
0.2998
```

- 0.1 has no finite binary expansion, so every format **rounds** it
- Addition is **not associative**: the order of a sum changes the result, so parallel sums are not bit-reproducible
- Fewer bits → **overflow** sooner, bigger rounding errors, and rounding errors **accumulate** over millions of training steps
- FP4 cannot tell 2.4 from 2.5, or 5 from 6

The fix is mixed precision: keep a **master copy** of the weights in FP32, run the big matrix multiplies in BF16 / FP8 / FP4 with block scales, and **accumulate** sums in FP32. The 2018 recipe gave 2–3× at the same accuracy; DeepSeek-V3 (2024) trained in FP8; NVFP4 training arrived in 2025.

Micikevicius et al., "Mixed Precision Training", ICLR 2018; DeepSeek-V3 technical report, Dec 2024; NVIDIA, "Pretraining Large Language Models with NVFP4", Sept 2025.



Practice: store 0.3 in FP8 E4M3

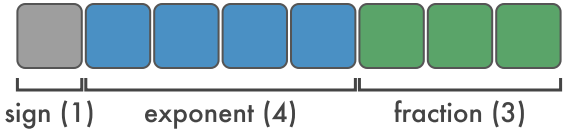
FP8 E4M3: 1 sign bit, 4 exponent bits (bias 7), 3 fraction bits

$$\text{value} = (-1)^{\text{sign}} \times 2^{\text{exponent} - 7} \times (1 + \text{fraction} / 8)$$

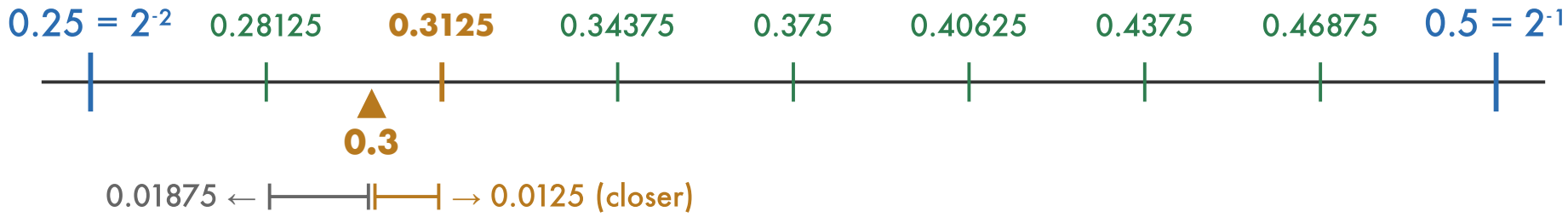
1. What bit pattern does 0.3 get?
2. What is the **closest representable number**?
3. What is the **error**, absolute and relative?

Hint: start by writing 0.3 as $1.xxx \times 2^{e}$, the same way we encoded 0.625 in the 6-bit float.

Practice: answer



- **0.3 = 1.2 × 2^{-2}**: exponent −2 + 7 = 5 = **0101**, so 0.3 lies between 2^{-2} and 2^{-1}
- In that interval the 3 fraction bits give **8 steps of 1/32**; 1.2 × 8 = 9.6 is not an integer, so 0.3 falls between two of them



Closest: 0.3125 = 0 0101 010. Error 0.0125, about 4.2% of 0.3

- The same 0.3 in FP32 is off by 4 × 10^{-8}, in FP16 by 1.6 × 10^{-4}, in BF16 by 2.6 × 10^{-3}; in FP4 it becomes 0.5 (67% off)

Q4: How can we make copyij even faster?

- Once the access pattern is right, the copy is **bandwidth-bound**: time = bytes ÷ bandwidth
- Levers: fewer **bytes per number**, SIMD, more cores or a GPU, or no copy at all
- A float is scientific notation in binary: **sign, exponent (range), fraction (precision)**; only the widths differ from FP64 to FP4
- Halving the bits halves the bytes moved, doubles Tensor Core throughput, and lets models fit one level higher in the hierarchy: **FP4 runs 125× FP32 on Blackwell**
- The price is precision: rounding, overflow, non-associativity; **mixed precision** and **block scaling** keep the error in check
- Choosing the number format is a systems decision: **speed, memory, and cost vs. accuracy**

Summary: four questions

1. **How to measure the speed of a program?** Time the part you care about, repeat, report the hardware. Time = instructions × cycles per instruction × seconds per cycle, and the bottleneck is compute, parallelism, or data movement
2. **Why is C faster than Python? Why is NumPy faster?** C compiles to a few machine instructions per element; Python runs ~250 cycles of interpreter work per element; NumPy pays the interpreter once per call and runs an optimized C loop
3. **Why is copyij faster than copyji?** Memory is ~150× slower than L1, so the hierarchy caches lines and prefetches; copyij walks memory in order and has locality, copyji does not
4. **How can we make copyij even faster?** Move fewer bytes: smaller number formats, SIMD, parallel workers, or no copy. Fewer bits per number is why GPUs are fast at ML, at the cost of precision

Everything in this course comes back to the same idea: moving data is the cost; organize the computation so that the data you need is close and small.